Laporan Tugas Kecil 3

Rush Hour Puzzle Solver dengan Pathfinding

Disusun untuk memenuhi tugas mata kuliah IF2211 Strategi Algoritma pada Semester 2 (Genap) Tahun Akademik 2024/2025

Disusun oleh:

Razi Rachman Widyadhana	13523004
Muhammad Adha Ridwan	13523098

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2025

Daftar Isi

Bab I: Deskripsi Masalah **1**

Bab II: Algoritma Pathfinding pada Rush Hour Puzzle **3**
2.1. Algoritma *Pathfinding* . . . 3
2.2. Uniform Cost Search (UCS) . . . 4
2.3. Greedy Best-First Search (GBFS) . . . 6
2.4. A* Search . . . 8
2.5. Fringe Search . . . 10
2.6. Heuristik yang digunakan . . . 13
2.6.1 Manhattan Distance . . . 13
2.6.2 Blocking Cars . . . 13
2.7. Analisis Algoritma *Pathfinding* dalam Penyelesaian Rush Hour Puzzle . . . 14
2.8. Algoritma Penyelesaian Rush Hour Puzzle . . . 17

Bab III: Implementasi Program dengan `Java` **18**
3.1. `Car.java` . . . 18
3.2. `Board.java` . . . 21
3.3. `State.java` . . . 32
3.4. `Algorithm.java` . . . 39
3.4.1 `UCS.java` . . . 41
3.4.2 `GBFS.java` . . . 42
3.4.3 `AStar.java` . . . 45
3.4.4 `Fringe.java` . . . 48
3.5. Penjelasan Bonus yang diimplementasikan . . . 52
3.5.1 Algoritma Alternatif . . . 52
3.5.2 Heuristik Alternatif . . . 52
3.5.3 *Graphical User Interface* . . . 52

Bab IV; Eksperimen **54**
4.1. Eksperimen . . . 54
4.2. Analisis dan Pembahasan . . . 75
4.2.1 Uniform Cost Search . . . 75
4.2.2 Greedy Best First Search . . . 75
4.2.3 A* Search . . . 76
4.2.4 Fringe Search . . . 76

Bab V: Penutup **77**
5.1. Kesimpulan . . . 77
5.2. Saran . . . 77
5.3. Tautan . . . 77
5.4. Lampiran . . . 78

Referensi **79**

Bab I
Deskripsi Masalah

Rush Hour adalah sebuah permainan *puzzle* logika berbasis grid yang menantang pemain untuk menggeser kendaraan di dalam sebuah kotak (biasanya berukuran 6x6) agar mobil utama (biasanya berwarna merah) dapat keluar dari kemacetan melalui pintu keluar di sisi papan. Setiap kendaraan hanya bisa bergerak lurus ke depan atau ke belakang sesuai dengan orientasinya (horizontal atau vertikal), dan tidak dapat berputar. Tujuan utama dari permainan ini adalah memindahkan mobil merah ke pintu keluar dengan jumlah langkah seminimal mungkin.

Gambar 1: Rush Hour Puzzle (Sumber: Think Fun)

Komponen penting dari permainan Rush Hour terdiri dari:

1. **Papan** – Papan merupakan tempat permainan dimainkan.

   Papan terdiri atas *cell*, yaitu sebuah *singular point* dari papan. Sebuah piece akan menempati *cell-cell* pada papan. Ketika permainan dimulai, semua *piece* telah diletakkan di dalam papan dengan konfigurasi tertentu berupa lokasi *piece* dan orientasi, antara horizontal atau vertikal.

   Hanya *primary piece* yang dapat digerakkan keluar papan melewati pintu keluar. *Piece* yang bukan *primary piece* tidak dapat digerakkan keluar papan. Papan memiliki satu pintu keluar yang pasti berada di dinding papan dan sejajar dengan orientasi *primary piece*.

2. **Piece** – *Piece* adalah sebuah kendaraan di dalam papan.

   Setiap *piece* memiliki posisi, ukuran, dan orientasi. Orientasi sebuah *piece* hanya dapat berupa horizontal atau vertikal–tidak mungkin diagonal. *Piece* dapat memiliki beragam ukuran, yaitu jumlah cell yang ditempati oleh *piece*. Secara standar, variasi ukuran sebuah *piece* adalah 2-*piece* (menempati 2 *cell*) atau 3-*piece* (menempati 3 *cell*). Suatu *piece* tidak dapat digerakkan melewati/menembus *piece* yang lain.

3. **Primary Piece**

   Primary piece adalah kendaraan utama yang harus dikeluarkan dari papan (biasanya berwarna merah). Hanya boleh terdapat satu *primary piece*.

4. **Pintu Keluar**

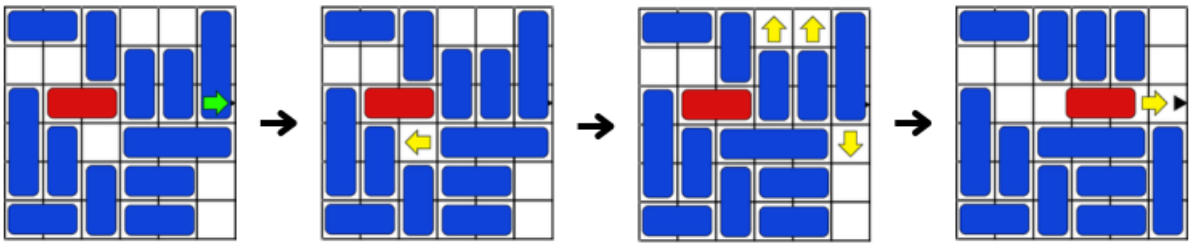
   Pintu keluar adalah tempat *primary piece* dapat digerakkan keluar untuk menyelesaikan permainan

5. **Gerakan**

   Gerakan yang dimaksudkan adalah pergeseran *piece* di dalam permainan. *Piece* hanya dapat bergerak/bergeser lurus sesuai orientasinya (atas-bawah jika vertikal dan kiri-kanan jika horizontal). Suatu *piece* tidak dapat digerakkan melewati/menembus *piece* yang lain.

Ilustrasi kasus:

- Diberikan sebuah papan berukuran 6 x 6 dengan 12 *piece* kendaraan dengan 1 *piece* merupakan *primary piece*.
- Pemain dapat menggeser-geser *piece* (termasuk *primary piece*) untuk membentuk jalan lurus antara *primary piece* dan pintu keluar.
- Puzzle dinyatakan telah selesai apabila *primary piece* dapat digeser keluar papan melalui pintu keluar.



Gambar 2: Ilustrasi Kasus Penyelesaian Puzzle (Sumber: Dokumen Spesifikasi Tugas Kecil 3 Stima 2024/2025)

Pada tugas kecil ini, para mahasiswa peserta mata kuliah IF2211 Semester Genap tahun 2024/2025 diminta untuk program sederhana dalam bahasa `C/C++/Java/Javascript` yang mengimplementasikan algoritma pathfinding *Greedy Best First Search*, UCS (*Uniform Cost Search*), dan A* dalam menyelesaikan permainan Rush Hour.

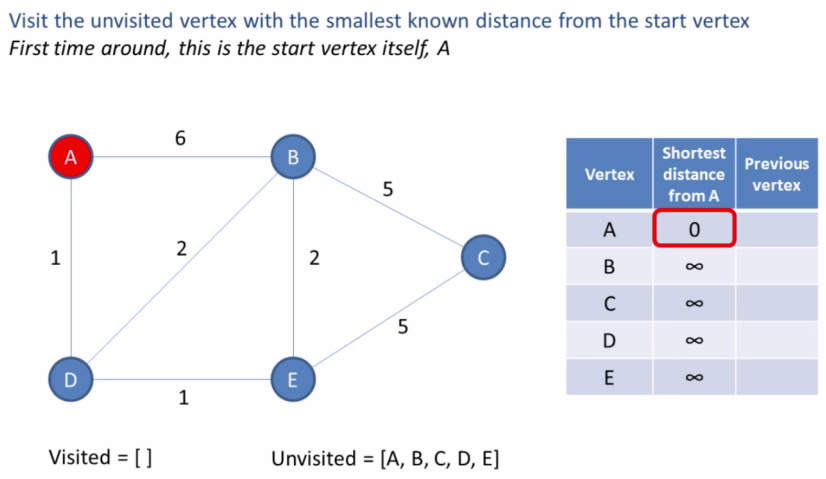
Bab II

Algoritma Pathfinding pada Rush Hour Puzzle

2.1. Algoritma *Pathfinding*

Algoritma ***Pathfinding*** adalah suatu metode yang digunakan untuk menemukan suatu jalur terpendek atau terefisien di dalam sebuah graf atau sebuah jaringan. Graf tersebut terdiri dari simpul-simpul (*nodes*) yang dihubungkan oleh sisi-sisi (*edges*).

Dalam implementasi teknis, *pathfinding* sering kali direpresentasikan sebagai graf, di mana setiap simpul (*node*) mewakili titik dalam ruang atau peta yang perlu dilalui. Setiap simpul menyimpan informasi penting seperti posisi (x, y), jarak atau biaya untuk mencapai simpul tersebut, dan perkiraan biaya menuju tujuan (jika menggunakan algoritma berbasis heuristik seperti A*). Informasi ini memungkinkan algoritma *pathfinding* untuk mengeksplorasi dan memilih jalur yang lebih efisien, menghindari jalur yang lebih panjang atau mahal.



Gambar 3: Contoh Algoritma Pathfinding

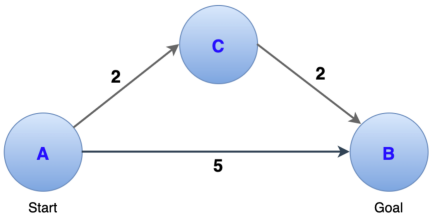
Dalam konteks permainan Rush Hour Puzzle, *pathfinding* digunakan untuk menentukan urutan pergerakan kendaraan agar mobil merah (kendaraan utama) dapat keluar dari area parkir dengan jumlah langkah minimal.

Pada Tugas Kecil 3 ini, digunakan Uniform Cost Search (UCS), Greedy Best First Search, A*, dan Fringe Search sebagai algoritma *pathfinding* pada penyelesaian Rush Hour Puzzle.

2.2. Uniform Cost Search (UCS)

Algoritma UCS adalah algoritma *uninformed search* yang dirancang untuk menemukan jalur dengan *cost* paling rendah dalam *weighted graph*. Berbeda dengan DFS atau BFS searches yang mengeksplorasi node berdasarkan urutan yang telah ditentukan, UCS memprioritaskan simpul berdasarkan *path cost* $g(n)$. Hal ini membuat UCS bersifat optimal karena selalu menemukan jalur terpendek menuju *goal* dalam hal *cost*, bukan hanya jarak atau kedalaman.

Dengan mempertimbangkan ilustrasi di bawah ini, jalur mana yang akan dipilih jika perlu bergerak dari titik A ke titik B, $A \rightarrow C \rightarrow B$ atau $A \rightarrow B$?



Gambar 4: Ilustrasi kasus UCS

Path cost dari A ke B adalah 5, sedangkan *path cost* dari A ke C ke B adalah 4 (2 + 2). Sesuai definisinya, UCS akan mempertimbangkan *path cost* yang paling rendah, yaitu 4, maka jalur $A \rightarrow C \rightarrow B$ akan dipilih dalam UCS.

Untuk menyimpan informasi urutan *path cost* mulai dari yang terendah dalam tahap eksplorasinya, UCS memanfaatkan struktur data *Priority Queue*. Dalam hal ini, makin kecil *path cost total* dari simpul awal ke simpul saat ini, maka makin menjadi prioritas simpul tersebut untuk dieksplorasi selanjutnya oleh UCS.

Secara algoritmik, tahapan UCS dapat diurai menjadi:

1. **Inisialisasi:** Masukkan simpul awal v yang diberikan ke dalam *Priority Queue* dan tandai sebagai telah dikunjungi dengan *cost*-nya biasanya diisi dengan 0.
2. **Ekspansi:** Selama *Priority Queue* tidak kosong:
   - Keluarkan simpul terdepan (memiliki *path cost* terendah) dari *Priority Queue* dan kunjungi simpul tersebut.
   - Kunjungi simpul w yang bertetangga dengan simpul v dan hitung *cumulative path cost* dengan menambahkan *cost* dari sisi menuju *neighbor* tersebut ke *cost* dari *path* ke simpul saat ini.
   - *Path cost* yang baru terakumulasi akan ditambahkan ke *Priority queue*, memastikan urutan selalu berdasarkan *cost*.
   - Ulangi sampai mencapai simpul u yang tidak memiliki tetangga yang belum dikunjungi (seluruhnya telah berada di *closedList*).
3. **Basis:** Pencarian berakhir bila tidak ada lagi simpul yang belum dikunjungi yang dapat dicapai dari simpul yang telah dikunjungi atau ditemukan simpul target melalui *path* dengan *cost* paling rendah.

Dalam konteks *pathfinding* dalam Rush Hour Puzzle, UCS akan mengeksplorasi semua kemungkinan pergerakan kendaraan dengan memprioritaskan langkah yang memiliki biaya lebih rendah, yaitu pergerakan yang lebih sedikit atau tidak memerlukan banyak kendaraan yang dipindahkan hingga menemukan solusi optimal, yaitu ketika mobil merah berhasil keluar dari area parkir dengan jumlah langkah minimal atau solusi tidak ditemukan.

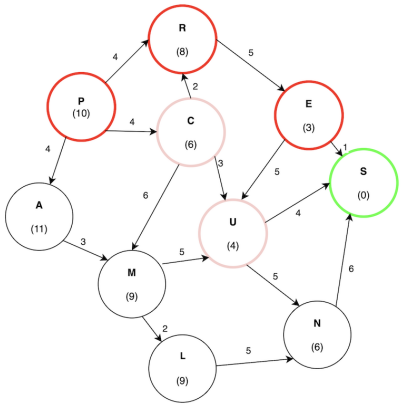
Berikut implementasinya untuk *pathfinding* pada Rush Hour dalam *pseudocode*:

```
procedure UniformCostSearch(input initialBoard, initialPieces) →
himpunan solusi (moves)
{ Mengembalikan daftar langkah (moves) untuk mencapai solusi }

Deklarasi
    MAX_COST         : integer
    visitedStates    : set of string
    openList         : priority queue of state
                       { board, pieces, stateString, cost, moves }
    cur, next        : record
                       { board, pieces, stateString, cost, moves }
Algoritma:
    MAX_COST      ← width(initialBoard) * height(initialBoard) * Multi
    visitedStates ← {}
    openList      ← empty priority queue

    { siapkan state awal }
    cur.board       ← initialBoard
    cur.pieces      ← initialPieces
    cur.stateString ← stringify(initialBoard)
    cur.cost        ← 0
    cur.moves       ← []
    openList.push(cur)

    while (pq != []) do
        cur ← openList.pop()   { ambil state dengan cost terkecil }

        if (cur.stateString not in visitedStates) then
            visitedStates ← visitedStates U { cur.stateString }

            if isSolved(cur.pieces) then return cur.moves endif
            if (cur.cost <= MAX_COST) then
                for move in getValidMoves(cur.board, cur.pieces) do
                    (next.board, next.pieces) ← applyMove(cur.board,
                                                cur.pieces, move)
                    next.stateString ← next.board.toString()

                    if (next.stateString not in visitedStates) then
                        next.cost  ← cur.cost + 1
                        next.moves ← cur.moves + [move]
                        openList.push(next)
                    endif
                endfor
            endif
        endif
    endwhile

    return []  { tidak ada solusi }
```

2.3. Greedy Best-First Search (GBFS)

Algoritma GBFS merupakan sebuah *informed search algorithm* yang memilih path untuk dieksplorasi berdasarkan *fungsi evaluasi* $f(n)$ dari simpul saat ini terhadap seberapa menjanjikannya dalam mencapai *goal* dan mengabaikan *edge weights* dalam *weighted graph* karena hanya *heuristic value* yang dipertimbangkan.

Heuristic value diperoleh melalui *heuristic function* sehingga $f(n) = h(n)$. Untuk menentukan *path* mana yang paling menjanjikan dengan mempertimbangkan *cost* dari *current path* dan *estimated cost* dari *remaining paths*. Jika *cost* dari *current path* lebih rendah daripada *estimated cost* dari *remaining paths*, maka *current path* dipilih. Proses ini diulang hingga *goal* tercapai.



Gambar 5: Ilustrasi kasus GBFS

Total *cost* untuk *path* ($P \rightarrow C \rightarrow U \rightarrow S$) adalah 11. Masalah potensial dengan GBFS terlihat pada path ($P \rightarrow R \rightarrow E \rightarrow S$) yang memiliki cost 10, lebih rendah daripada ($P \rightarrow C \rightarrow U \rightarrow S$). Akibat dari tidak mempertimbangkan biaya yang telah ditempuh, algoritma ini mungkin memilih *path* yang terlihat menjanjikan berdasarkan *heuristic* saja, meskipun memiliki *actual cost* yang lebih tinggi.

Secara algoritmik, tahapan GBFS dapat diurai menjadi:

1. **Inisialisasi:** Masukkan simpul awal v ke dalam *Priority Queue* dan tandai sebagai telah dikunjungi dengan inisialisasi nilai *heuristic* $h(v)$.
2. **Ekspansi:** Selama *priority queue* tidak kosong:
   - Keluarkan simpul v terdepan (memiliki nilai $h(v)$ terendah) dari *priority queue* dan kunjungi simpul tersebut.
   - Untuk setiap simpul tetangga w dari v, hitung nilai heuristik $h(w)$ menggunakan fungsi heuristik yang diberikan.
   - Tambahkan setiap w beserta nilai $h(w)$ ke dalam *priority queue*, tanpa memperhatikan *edge weight*.
   - Ulangi sampai mencapai simpul u yang tidak memiliki tetangga yang belum dikunjungi
3. **Basis:** Pencarian berakhir bila *priority queue* kosong (tidak ditemukan jalan ke goal) atau goal tercapai (jika simpul yang dipilih adalah tujuan).

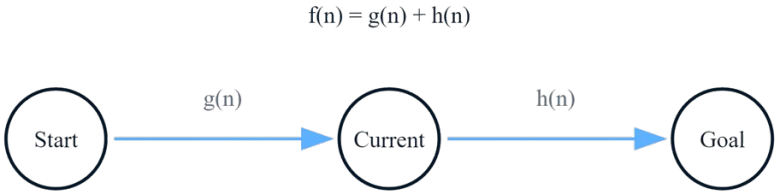
Dalam Rush Hour Puzzle, Greedy Best First Search akan mengeksplorasi semua kemungkinan pergerakan kendaraan dengan terus memilih kenderaan yang sepertinya paling dekat dengan pintu keluar berdasarkan heuristiknya hingga menemukan solusi, yaitu ketika mobil merah (*primary*) berhasil keluar dari area parkir atau solusi tidak ditemukan (*openList* telah kosong).

Berikut implementasinya untuk *pathfinding* pada Rush Hour dalam *pseudocode*:

```
procedure GreedyBestFirstSearch(input initialBoard, initialPieces) →
himpunan solusi (moves)
{ Mengembalikan daftar langkah (moves) untuk mencapai solusi
menggunakan Greedy Best-First Search }

Deklarasi
    MAX_COST     : integer
    visitedStates: set of string
    openList     : priority queue of state
                   {board, pieces, stateString, heuristic, cost, moves}
    cur, next    : record
                   {board, pieces, stateString, heuristic, cost, moves}
Algoritma:
    MAX_COST      ← width(initialBoard) * height(initialBoard) * Multi
    visitedStates ← {}
    openList      ← empty priority queue

    { siapkan state awal }
    openList.push(cur)

    while (openList != []) do
        cur ← openList.pop()  { ambil state dengan heuristic terkecil }

        if (cur.stateString not in visitedStates) then
            visitedStates ← visitedStates U { cur.stateString }

            if isSolved(cur.pieces) then return cur.moves endif

            if (cur.cost <= MAX_COST) then
                for move in getValidMoves(cur.board, cur.pieces) do
                    (next.board, next.pieces) ← applyMove(cur.board,
                    cur.pieces, move)
                    next.stateString ← next.board.toString()

                    if (next.stateString not in visitedStates) then
                        next.cost      ← cur.cost + 1
                        next.heuristic ← heuristicFunc(next.board,
                        next.pieces)
                        next.moves     ← cur.moves + [move]
                        openList.push(next)
                    endif
                endfor
            endif
        endif
    endwhile

    return []  { tidak ada solusi }
```

2.4. A* Search

A* merupakan *informed search algorithm* yang menemukan *single-pair shortest path* antara simpul awal (*source*) dan simpul goal (*destination*) pada *weighted graph*. Algoritma ini tidak hanya mempertimbangkan *actual cost* dari simpul awal ke simpul saat ini, yaitu $g(n)$. Namun, juga mencoba mengestimasi *cost* yang akan dibutuhkan dari simpul saat ini ke simpul goal menggunakan *heuristics* $h(n)$. Kemudian, algoritma memilih simpul dengan $f(n) = g(n) + h(n)$ terendah sebagai simpul selanjutnya hingga mencapai simpul goal.



Gambar 6: Ilustrasi $f(n)$ pada A*

Oleh karena itu, A* merupakan salah satu algoritma pencarian jalur yang efisien dan sering digunakan untuk menemukan jalur terpendek dalam *weighted graph* dengan menggabungkan dua komponen utama untuk menentukan jalur terbaik.

Menggunakan *heuristic* yang baik penting dalam menentukan *performance* A*. Nilai $h(n)$ idealnya sama dengan *exact cost* untuk mencapai *destination*. Namun, hal ini tidak mungkin ketika tidak mengetahui *path*-nya. Dapat dipilih metode yang memberikan nilai *exact* beberapa kali, misalnya ketika *traveling in a straight line* tanpa *obstacles*. Ini akan menghasilkan *performance* A* yang sempurna dalam kasus tersebut.

Secara algoritmik, tahapan A* dapat diurai menjadi:

1. **Inisialisasi:** Masukkan simpul awal v ke dalam *priority queue* dan tandai sebagai telah dikunjungi. Inisialisasi nilai $g(v) = 0$ dan $h(v)$ sesuai fungsi heuristik; hitung $f(v) = g(v) + h(v)$.
2. **Ekspansi:** Selama *priority queue* tidak kosong:
   - Keluarkan simpul v terdepan (memiliki nilai $f(v)$ terendah) dari *priority queue* dan kunjungi simpul tersebut.
   - Untuk setiap simpul tetangga w dari v:
     - Hitung biaya kumulatif baru $g(w) = g(v) + \text{cost}(v, w)$.
     - Hitung heuristik $h(w)$ dengan fungsi heuristik yang diberikan.
     * Hitung nilai evaluasi $f(w) = g(w) + h(w)$.
     - Jika w belum dikunjungi atau ditemukan rute dengan $f(w)$ lebih rendah, tambahkan atau perbarui entri w beserta nilai $g(w)$, $h(w)$, dan $f(w)$ di dalam *priority queue*.
3. **Basis:** Pencarian berakhir bila *priority queue* kosong (tidak ditemukan jalan ke goal) atau simpul tujuan dipilih (goal tercapai).

Dalam Rush Hour Puzzle, A* akan memilih langkah yang tidak hanya mengurangi jarak yang telah ditempuh tetapi juga memperkirakan berapa jauh lagi kendaraan tersebut dari pintu keluar hingga menemukan solusi optimal, yaitu ketika mobil merah (*primary)* berhasil keluar dari area parkir dengan jumlah langkah minimal atau solusi tidak ditemukan.

Berikut implementasi A* untuk *pathfinding* pada Rush Hour dalam *pseudocode*:

```
procedure AStarSearch(input initialBoard, initialPieces) →
himpunan solusi (moves)
{ Mengembalikan daftar langkah (moves) untuk mencapai solusi
menggunakan A* Search }

Deklarasi
MAX_COST      : integer
visitedStates : set of string
openList      : priority queue of state
                {board, pieces, stateString, cost, heuristic, f, moves}
cur, next     : record
                {board, pieces, stateString, cost, heuristic, f, moves}
Algoritma:
    MAX_COST      ← width(initialBoard) * height(initialBoard) * Multi
    visitedStates ← {}
    openList      ← empty priority queue

    { siapkan state awal }
    openList.push(cur)

    while (openList != []) do
        cur ← openList.pop()   { ambil state dengan f(n) terkecil }

        if (cur.stateString not in visitedStates) then
            visitedStates ← visitedStates U { cur.stateString }

            if isSolved(cur.pieces) then return cur.moves endif

            if (cur.cost <= MAX_COST) then
                for move in getValidMoves(cur.board, cur.pieces) do
                    (next.board, next.pieces) ← applyMove(cur.board,
                    cur.pieces, move)
                    next.stateString ← next.board.toString()

                    if (next.stateString not in visitedStates) then
                        next.cost      ← cur.cost + 1
                        next.heuristic ← heuristicFunc(next.board,
                        next.pieces)
                        next.f         ← next.cost + next.heuristic
                        next.moves     ← cur.moves + [move]
                        openList.push(next)
                    endif
                endfor
            endif
        endif
    endwhile

    return []  { tidak ada solusi }
```

2.5. Fringe Search

Fringe Search merupakan alternatif dari algoritma A* yang lebih efisien dalam hal penggunaan memori. Algoritma ini menggabungkan konsep dari A* dengan *iterative deepening* untuk mengontrol ekspansi simpul. Seperti A*, Fringe Search juga menggunakan fungsi evaluasi $f(n) = g(n) + h(n)$, di mana $g(n)$ adalah *actual cost* dari simpul awal ke simpul saat ini, dan $h(n)$ adalah estimasi *cost* dari simpul saat ini ke simpul goal berdasarkan *heuristic*.

Perbedaan utama antara Fringe Search dan A* terletak pada struktur data yang digunakan dan cara eksplorasi *search space*. Daripada menggunakan *priority queue*, Fringe Search menggunakan daftar biasa (*list*) dan membatasi eksplorasi simpul berdasarkan nilai batas *f-limit* yang ditingkatkan secara bertahap. Pendekatan ini memungkinkan algoritma untuk mengurangi overhead memori dan kompleksitas yang terkait dengan operasi *priority queue*.

Gambar 7: Ilustrasi eksplorasi simpul dengan Fringe Search menggunakan f-limit

Fringe Search bekerja dengan menjelajahi semua simpul dengan nilai $f(n)$ hingga batas *f-limit* saat ini, kemudian meningkatkan batas tersebut ke nilai $f(n)$ terendah berikutnya yang melebihi batas sebelumnya. Proses ini berulang hingga solusi ditemukan atau tidak ada lagi simpul yang dapat dijelajahi.

Efisiensi Fringe Search bergantung pada kualitas fungsi *heuristic* yang digunakan. Seperti halnya A*, *heuristic* yang baik akan mempercepat pencarian dengan memberikan estimasi yang lebih akurat tentang jarak ke simpul goal. Dalam kondisi optimal, Fringe Search dapat mencapai kinerja yang setara dengan A* namun dengan kebutuhan memori yang lebih rendah.

Secara algoritmik, tahapan Fringe Search dapat diurai menjadi:

1. **Inisialisasi:** Masukkan simpul awal v ke dalam *fringe* (daftar simpul) dan tandai sebagai telah dikunjungi. Inisialisasi nilai $g(v) = 0$ dan $h(v)$ sesuai fungsi heuristik; hitung $f(v) = g(v) + h(v)$. Tetapkan nilai awal *f-limit* $= f(v)$.
2. **Ekspansi dengan Batasan:** Selama *fringe* tidak kosong:
   - Urutkan *fringe* berdasarkan nilai $f(n)$ berdasarkan urutan terkecil.
   - Inisialisasi $nextFLimit = \infty$ dan $nextFringe = []$.
   - Untuk setiap simpul v dalam *fringe*, Jika $f(v) >$ *f-limit* perbarui $nextFLimit = \min(nextFLimit, f(v)$ dan tambahkan v ke *nextFringe*, Jika tidak ($f(v) \leq$ *f-limit*), kunjungi simpul tersebut:
     * Jika v adalah simpul goal, kembalikan jalur dari simpul awal ke v.
     * Untuk setiap simpul tetangga w dari v:

1. Hitung biaya kumulatif baru $g(w) = g(v) + \text{cost}(v, w)$.
2. Hitung heuristik $h(w)$ dengan fungsi heuristik yang diberikan.
3. Hitung nilai evaluasi $f(w) = g(w) + h(w)$.
4. Jika w belum dikunjungi:
   (a) Tandai w sebagai telah dikunjungi.
   (b) Jika $f(w) \leq$ *f-limit*, tambahkan w ke *nextFringe*.
   (c) Jika tidak, perbarui *nextFLimit* $= \min(\textit{nextFLimit}, f(w))$ dan tambahkan w ke *nextFringe*.

3. **Perbarui Batas:** Setelah semua simpul dalam *fringe* diproses:
   - Jika *nextFringe* kosong dan *nextFLimit* $= \infty$, kembalikan bahwa tidak ada solusi.
   - Jika tidak, tetapkan *f-limit* = *nextFLimit* dan *fringe* = *nextFringe*, kemudian lanjutkan ke langkah 2.

Dalam Rush Hour Puzzle, Fringe Search akan mengeksplorasi status-status permainan yang memiliki nilai $f(n)$ di bawah atau sama dengan *f-limit* saat ini, kemudian meningkatkan batas tersebut secara bertahap hingga menemukan solusi optimal atau menyimpulkan bahwa tidak ada solusi

Berikut implementasi Fringe Search untuk Rush Hour dalam *pseudocode*:

```
procedure FringeSearch(input initialBoard, initialPieces) →
himpunan solusi (moves)
{ Mengembalikan daftar langkah (moves) untuk mencapai solusi
menggunakan Fringe Search }

Deklarasi
MAX_COST      : integer
visitedStates : set of string
fringe        : list of state
                {board, pieces, stateString, cost, heuristic, f, moves}
nextFringe    : list of state
cur           : record
                {board, pieces, stateString, cost, heuristic, f, moves}
fLimit        : integer
nextFLimit    : integer

Algoritma:
    MAX_COST      ← width(initialBoard) * height(initialBoard) * Multi
    visitedStates ← {}
    fringe        ← []

    { siapkan state awal }
    cur.board      ← initialBoard
    cur.pieces     ← initialPieces
    cur.stateString ← cur.board.toString()
    cur.cost       ← 0
    cur.heuristic  ← heuristicFunc(cur.board, cur.pieces)
    cur.f          ← cur.cost + cur.heuristic
    cur.moves      ← []
```

```
    visitedStates ← visitedStates U { cur.stateString }
    fringe.add(cur)
    fLimit ← cur.heuristic


    while (fringe != []) do
        sort(fringe) by f value  { urutkan berdasarkan nilai f }
        nextFLimit ← infinity
        nextFringe ← []

        for each state in fringe do
            cur ← state

            if (cur.f > fLimit) then
                nextFLimit ← min(nextFLimit, cur.f)
                nextFringe.add(cur)
                continue
            endif

            if isSolved(cur.pieces) then return cur.moves endif

            if (cur.cost <= MAX_COST) then
                for move in getValidMoves(cur.board, cur.pieces) do
                    (next.board, next.pieces) ← applyMove(cur.board,
                    cur.pieces, move)
                    next.stateString ← next.board.toString()

                    if (next.stateString not in visitedStates) then
                        visitedStates ← visitedStates U {next.stateStr
                        ing}

                        next.cost      ← cur.cost + 1
                        next.heuristic ← heuristicFunc(next.board,
                        next.pieces)
                        next.f         ← next.cost + next.heuristic
                        next.moves     ← cur.moves + [move]

                        if (next.f <= fLimit) then
                            nextFringe.add(next)
                        else
                            nextFLimit ← min(nextFLimit, next.f)
                            nextFringe.add(next)
                        endif
                    endif
                endfor
            endif
        endfor

        if (nextFringe = [] AND nextFLimit = infinity) then
            return []  { tidak ada solusi }
        endif

        fLimit ← nextFLimit
        fringe ← nextFringe
    endwhile

    return []  { tidak ada solusi }
```

2.6. Heuristik yang digunakan

Telah di-*mention* terkait fungsi heuristik $h(n)$ pada penyelesaian Rush Hour Puzzle, lantas bagaimana implementasi dan integrasinya ke algoritma *pathfinding* yang menggunakan $h(n)$ ke dalam penyelesaian Rush Hour Puzzle?

Pada Tugas Kecil 3 ini, diimplementasikan 2 heuristik yang digunakan untuk memperkiraan biaya dari simpul saat ini ke simpul tujuan. Berikut penjelasannya masing-masing secara lebih komprehensif:

2.6.1 Manhattan Distance

Jarak Manhattan merupakan metrik spasial yang menghitung jumlah total panjang segmen garis antara dua titik diukur terhadap setiap sumbu dalam sistem koordinat Kartesius.

Secara matematis, untuk Rush Hour Puzzle dengan orientasi horizontal, jarak Manhattan diformulasikan sebagai selisih kolom antara bagian terdepan kendaraan utama dengan sel tujuan pada baris yang sama. Sementara untuk orientasi vertikal, jarak dihitung sebagai selisih baris antara bagian terdepan mobil merah dengan pintu keluar pada kolom yang sama.

$$d = |x_p - x_{exit}| + |y_p - y_{exit}|$$

Dalam konteks pemecahan masalah pencarian, jarak Manhattan berfungsi sebagai instrumen heuristik yang memberikan estimasi biaya perjalanan dalam ruang keadaan diskrit.

Implementasi fungsi heuristik $h(n)$ berbasis Manhattan pada Rush Hour Puzzle menghitung jumlah langkah minimal yang diperlukan secara horizontal atau vertikal dari posisi mobil merah (*primary piece*) pada keadaan saat ini menuju pintu keluar (*exit goal*), tanpa memperhitungkan keberadaan kendaraan-kendaraan penghalang.

2.6.2 Blocking Cars

Blocking Cars adalah sebuah heuristik yang menghitung jumlah kendaraan yang secara langsung menghalangi jalur pergerakan *primary piece* menuju pintu keluar. Dalam penyelesaian Rush Hour Puzzle, heuristik ini menilai berapa banyak kendaraan lain yang menempati petak-petak di antara posisi *primary piece* dan pintu keluar pada baris yang sama. Dengan demikian, heuristik Blocking Cars memberikan estimasi awal tentang seberapa banyak langkah perpindahan yang diperlukan sebelum jalur menuju goal benar-benar terbuka.

Heuristik ini sangat berguna karena mencerminkan tingkat kesulitan sejati suatu konfigurasi papan: semakin banyak kendaraan yang menghalangi, semakin kompleks pula solusi yang harus dicapai. Meskipun tidak memperhitungkan perpindahan kendaraan tambahan di luar garis langsung menuju exit, Blocking Cars tetap menjadi indikator penting dalam memilih simpul yang paling menjanjikan untuk diekspansi dalam algoritma pencarian, sehingga mempercepat proses menemukan jalur keluar optimal.

2.7. Analisis Algoritma *Pathfinding* dalam Penyelesaian Rush Hour Puzzle

Setelah membahas masing-masing definisi dari algoritma yang akan digunakan sebagai implementasi penyelesaian, bagaimana caranya menggunakan algoritma tersebut ke dalam penyelesaian Rush Hour Puzzle?

Dalam strategi pemecahan masalah, fokus pertama yang dilakukan adalah bagaimana merepresentasikan persoalan menjadi State yang optimal. Setiap konfigurasi papan diabstraksikan sebagai struktur data yang mampu mencatat pintu keluar dan posisi setiap kendaraan pada State tersebut.

Pendekatan ini memungkinkan pembuatan operasi `move` yang menghasilkan keadaan baru dengan tetap mudah diidentifikasi dan dibandingkan pergerakannya. Setelah representasi papan permainan terbentuk, tahap implementasi algoritma *pathfinding* seperti UCS, Greedy Best-First, dan A* untuk menjelajahi ruang State tersebut dengan tujuan menemukan *path* (sekuens langkah yang memungkinkan kendaraan utama mencapai pintu keluar).

Pada setiap algoritma pencarian jalur, terdapat penggunaan fungsi evaluasi $f(n)$ yang berperan dalam menentukan urutan prioritas eksplorasi simpul. Fungsi $g(n)$ merepresentasikan biaya perjalanan dari simpul awal sampai ke simpul yang sedang dipertimbangkan atau simpul n. Perbedaan mendasar di antara berbagai algoritma pencarian terletak pada cara pendefinisian $f(n)$ tersebut.

- $f(n)$:

  Fungsi evaluasi yang memperkirakan dan memprioritaskan *total cost* untuk mencapai simpul n pada *frontier* (terurut dari yang terendah).

- $g(n)$:

  Biaya yang telah ditempuh dari titik awal menuju simpul n. Digunakan pada UCS dan A*.

- $h(n)$:

  Perkiraan biaya dari simpul n menuju tujuan (heuristik). Digunakan pada GBFS dan A*.

Terdapat konsep penting jika membahas tentang heuristik. Suatu heuristik dalam algoritma A* disebut *admissible* bila tidak pernah memberikan estimasi yang melebihi biaya aktual dari simpul saat ini menuju simpul tujuan (bersifat optimistik).

$$h(n) \leq h^*(n), \quad \text{untuk setiap node } n$$

Dalam hal ini, $h(n)$ merupakan perkiraan *cost* dari simpul n ke tujuan dan $h^*(n)$ adalah *true cost* dari node n ke tujuan. Untuk masing-masing heuristik yang digunakan, akan dianalisis apakah heuristik tersebut *admissible* atau tidak.

`Manhattan Distance` mengkalkulasi jarak antara kendaraan utama dengan pintu keluar tanpa memperhitungkan keberadaan penghalang disebut *admissible* karena senantiasa menghasilkan estimasi yang tidak lebih tinggi dari biaya sebenarnya.

Sementara itu, `Blocking Cars` yang melakukan penghitungan terhadap banyaknya kendaraan yang menghalangi jalur keluar, juga disebut *admissible* karena hanya menghitung kuantitas penghalang, bukan biaya pemindahannya.

Mengingat setiap kendaraan penghalang minimal membutuhkan satu gerakan untuk dipindahkan, nilai

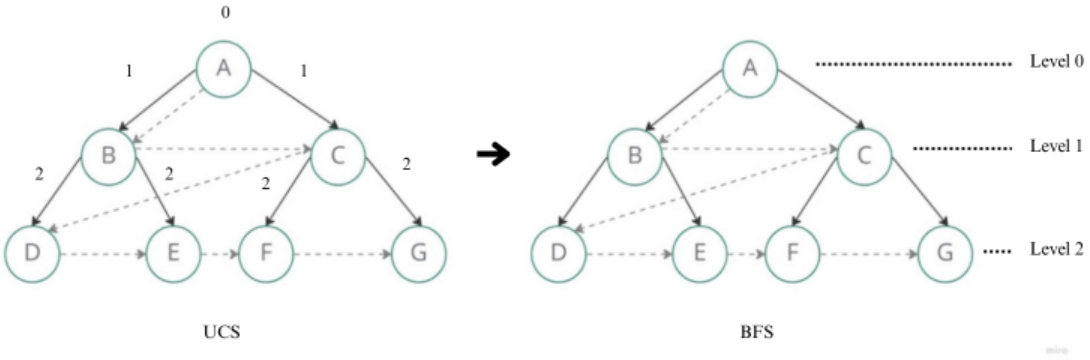
$$h(n) = \text{jumlah penghalang}$$

tidak akan menghasilkan estimasi yang melebihi jumlah langkah riil yang diperlukan untuk menyingkirkan seluruh penghalang tersebut. Dengan demikian, untuk tiap kendaraan penghalang, dipastikan dibutuhkan minimal satu langkah, tidak mungkin kurang dari itu.

Kemudian, terdapat hal yang menarik terkait UCS dan BFS dalam konteks penyelesaian Rush Hour Puzzle ini. Secara teoritis, algoritma Uniform Cost Search (UCS) dan Breadth-First Search (BFS) sebenarnya didasarkan pada konsep yang berbeda meskipun dalam kasus tertentu dapat menghasilkan perilaku yang serupa.

UCS dirancang untuk menemukan jalur berbiaya minimum dalam graf berbobot dengan memprioritaskan ekspansi node berdasarkan total biaya perjalanan, sedangkan BFS mengeksplorasi semua node secara level per level tanpa mempertimbangkan biaya perjalanan. Perbedaan fundamental ini menjadi signifikan dalam graf berbobot tidak seragam di mana UCS dapat menemukan solusi optimal sementara BFS tidak selalu demikian.

Dalam konteks penyelesaian permainan Rush Hour, setiap pergerakan kendaraan memiliki biaya yang sama berupa 1 sehingga algoritma Uniform Cost Search (UCS) dan Breadth-First Search (BFS) akan menghasilkan jalur solusi dengan panjang yang identik dan akan melakukan eksplorasi simpul berdasarkan jumlah langkah yang bertambah secara kumulatif.



Gambar 8: UCS dan BFS pada penyelesaian Rush Hour Puzzle

Hal ini terjadi karena UCS melakukan ekspansi simpul berdasarkan total biaya dari titik awal $g(n)$, yang nilainya sama dengan kedalaman simpul pada BFS ketika semua langkah bernilai 1.

Oleh karena itu, jalur yang dihasilkan oleh UCS dan BFS akan setara dalam hal jumlah langkah dan kualitas solusi. Lebih lanjut, karena urutan ekspansi dalam kedua algoritma tersebut mengikuti peningkatan nilai $g(n)$, maka urutan simpul yang dihasilkan juga akan sama apabila tidak terdapat mekanisme *tie-breaking* yang membedakan antara dua simpul dengan biaya yang identik.

Pada bagian sebelumnya, telah dibahas masing-masing UCS dan A*. Lantas algoritma mana yang lebih optimal? Secara teoritis, algoritma A* akan menunjukkan efisiensi yang lebih tinggi dibandingkan UCS dalam penyelesaian permainan Rush Hour melalui penggunaan fungsi heuristik yang mengarahkan pencarian ke arah solusi. Berbeda dengan UCS yang hanya bergantung pada fungsi $g(n)$ untuk mengeksplorasi node berdasarkan biaya kumulatif dari node awal, A* memanfaatkan fungsi evaluasi

$$f(n) = g(n) + h(n)$$

yang menggabungkan biaya perjalanan dengan estimasi jarak ke tujuan.

Penambahan komponen heuristik $h(n)$ ini memungkinkan A* untuk memprioritaskan eksplorasi node yang lebih menjanjikan, sehingga secara signifikan mengurangi jumlah node yang perlu dieksplorasi. Dengan karakteristik ini, A* mampu menemukan solusi optimal dengan eksplorasi ruang pencarian yang jauh lebih terarah dan efisien dibandingkan eksplorasi menyeluruh yang dilakukan UCS.

Penerapan heuristik *admissible* pada A* menjamin optimalitas solusi sambil meningkatkan efisiensi pencarian. Heuristik `Manhattan Distance` yang mengukur jarak dari kendaraan utama ke pintu keluar tanpa mempertimbangkan penghalang, serta heuristik `Blocking Car` yang menghitung jumlah kendaraan penghalang jalur keluar, keduanya telah dibuktikan bersifat *admissible* pada bagian sebelumnya.

Keunggulan efisiensi A* terhadap UCS menjadi semakin terlihat pada kasus Rush Hour yang kompleks dengan jumlah kendaraan dan konfigurasi papan yang rumit. Pada kasus tersebut, UCS cenderung mengeksplorasi terlalu banyak node yang tidak relevan dengan solusi karena tidak memiliki mekanisme untuk memprioritaskan arah eksplorasi. Sebaliknya, A* dengan heuristik yang tepat akan langsung mengarah ke solusi dengan meminimalkan eksplorasi node yang tidak produktif.

Hal ini menghasilkan penurunan drastis pada waktu komputasi dan penggunaan memori, sementara tetap mempertahankan jaminan menemukan solusi optimal selama heuristik yang digunakan bersifat *admissible*.

Lantas bagaimana dengan algoritma Greedy Best-First Search? GBFS tidak menjamin solusi optimal dalam penyelesaian masalah pencarian jalur seperti Rush Hour. Berbeda dengan algoritma A* dan UCS yang menghasilkan solusi optimal, Greedy Best-First Search hanya menggunakan fungsi heuristik h(n) sebagai fungsi evaluasinya tanpa mempertimbangkan biaya perjalanan g(n) dari simpul awal.

Pendekatan ini membuat algoritma tersebut mengeksplorasi secara eksklusif berdasarkan estimasi jarak ke tujuan, sehingga meskipun pencarian mungkin bergerak cepat menuju tujuan, jalur yang dihasilkan sering kali bukan merupakan jalur dengan biaya terendah.

Dalam konteks permainan Rush Hour, setiap pergerakan kendaraan memiliki biaya yang seragam. Greedy Best-First Search dengan heuristik `Manhattan Distance` atau `Blocking Cars` akan memprioritaskan eksplorasi simpul-simpul yang memiliki sedikit penghalang atau jarak yang dekat dengan pintu keluar tanpa mempertimbangkan berapa banyak langkah yang telah dilakukan sebelumnya. Meskipun pendekatan ini dapat mempercepat proses pencarian dengan mengurangi jumlah node yang perlu dieksplorasi, algoritma ini rentan terjebak/tertipu dalam jalur yang tampak menjanjikan di awal namun pada akhirnya membutuhkan lebih banyak langkah secara keseluruhan dibandingkan solusi optimal.

2.8. Algoritma Penyelesaian Rush Hour Puzzle

Secara garis besar dari definsi dan analisis yang telah disebutkan, dengan menggunakan struktur data *priority queue* diperolehlah metode general untuk menemukan solusi dari Rush Hour Puzzle secara algoritmik. Adapun algoritma *pathfinding* yang digunakan adalah sebagai berikut.

1. Masukkan *state* awal *board* v yang berisi seluruh kemungkinan gerakan tiap mobil yang valid ke dalam *priority queue* dan tandai sebagai telah dikunjungi. Inisialisasi nilai $f(n)$ ditentukan dengan algoritma *pathfinding* yang digunakan.
2. Selama *priority queue* tidak kosong:
   - Keluarkan state v terdepan (memiliki nilai $f(v)$ terendah) dari *priority queue* dan kunjungi state tersebut.
   - Untuk setiap state hasil kemungkinan gerakan tiap mobil w dari seluruh kemungkinan gerakan pada state v:
     - **UCS**:

       Kalkulasi fungsi evaluasi dengan biaya kumulatif baru $f(w) = g(w) = g(v) + \text{cost}(v, w)$.
     - **GBFS**:

       Kalkulasi fungsi evaluasi dengan heuristik melalui fungsi heuristik $f(w) = h(w)$ yang digunakan (Manhattan Distance ataupun Blocking Cars).
     - **A***:

       Kalkulasi terlebih dahulu $g(n)$ seperti **UCS** dan $h(n)$ seperti **GBFS** sehingga nilai evaluasi menjadi $f(w) = g(w) + h(w)$.
   - Jika w belum dikunjungi atau ditemukan rute dengan $f(w)$ lebih rendah, tambahkan atau perbarui entri w beserta nilai $f(w)$ di dalam *priority queue*.
3. Pencarian berakhir bila *priority queue* kosong (tidak ditemukan jalan ke pintu keluar) atau mobil merah (*primary car*) sampai ke pintu keluar (*goal* tercapai).

Bab III
Implementasi Program dengan Java

Algoritma yang telah dijelaskan pada Bab 2 diimplementasikan dengan bahasa pemrograman `Java`. Untuk menyesuaikan proses perancangan program yang *intended* pada `Java`, digunakan pendekatan *object-oriented programming*, di mana setiap komponen utama program akan dienkapulasi oleh *Object* dengan responsibilitasnya masing-masing. Untuk program ini, sebenarnya terdapat beberapa kelas. Akan tetapi, kelas utama adalah kelas yang melakukan komputasi *pathfinding* dan kelas representasi *board* permainan, untuk kelas yang bertanggung jawab komputasi adalah kelas `Algorithm` sebagai kelas abstrak, kemudian ada empat kelas yang melakukan implementasi diantaranya, `A*`, `UCS`, `GBFS`, `Fringe` dan kelas representasi *board* permainan adalah kelas `Board`, `Car` dan `State`.

Selain dari tujuh kelas yang digunakan, program kami juga menggunakan *Library* `JavaFX` untuk membuat *Graphical User Interface* dan *build tools* `Gradle` untuk membuat `.jar` dari program kami.

3.1. `Car.java`

```
package src.ADT;

/**
 * Class representing a car (vehicle) in the Rush Hour puzzle game.
 */

public class Car
{
    private char id;           // Character identifier for the car
    private int startRow;      // Starting row position
    private int startCol;      // Starting column position
    private int length;        // Number of cells the car occupies
    private int orientation;   // Orientation: true for horizontal,
    ↪  false for vertical
    private boolean isPrimary; // Whether this is the primary car
    ↪  that needs to reach the exit

    /**
     * Constructor for creating a car
     *
     * @param id Character identifier for the car
     * @param row Starting row position
     * @param col Starting column position
     * @param length Number of cells the car occupies
     * @param isPrimary Whether this is the primary car
     * @param orientation Orientation: true for horizontal, false
     ↪  for vertical
     */
```

```java
    public Car(char id,
               int row,
               int col,
               int length,
               boolean isPrimary,
               int orientation)
    {
        this.id          = id;
        this.startRow    = row;
        this.startCol    = col;
        this.length      = length;
        this.isPrimary   = isPrimary;
        this.orientation = orientation;
    }

    public char getId() { return this.id; }
    public void setId(char id) { this.id = id; }

    public int getStartRow() { return this.startRow; }
    public void setStartRow(int row) { this.startRow = row; }

    public int getStartCol() { return this.startCol; }
    public void setStartCol(int col) { this.startCol = col; }

    public int getLength() { return this.length; }
    public void setLength(int length) { this.length = length; }

    public boolean isPrimary() { return this.isPrimary; }
    public void setPrimary(boolean isPrimary) { this.isPrimary =
    ↪  isPrimary; }

    public int getOrientation() { return this.orientation; }
    public void setOrientation(int orientation) { this.orientation =
    ↪  orientation; }

    /**
     * Creates a copy of this car
     *
     * @return A new Car object with the same attributes
     */
    public Car copy()
    {
        return new Car(id, startRow, startCol, length, isPrimary,
        ↪  orientation);
    }

    /**
     * Move the car by the specified amount (positive or negative)
     * Positive means right/down, negative means left/up
     *
```

```java
     * @param amount Number of cells to move
     */
    public void move(int amount)
    {
        if (orientation == 1) this.startCol += amount;
        else this.startRow += amount;
    }

    /**
     * Get all the cells occupied by this car
     *
     * @return A 2D array where each element is [row, col] of an
     ↪  occupied cell
     */
    public int[][] getOccupiedCells()
    {
        int[][] cells = new int[length][2];

        for (int i = 0; i < length; i++)
        {
            if (orientation == 1)
            {
                cells[i][0] = this.getStartRow();
                cells[i][1] = this.getStartCol() + i;
            }
            else
            {
                cells[i][0] = this.getStartRow() + i;
                cells[i][1] = this.getStartCol();
            }
        }
        return cells;
    }

    /**
     * Get the string representation of the car
     *
     * @return A string representing the car
     */
    @Override
    public String toString()
    {
        return "Car [id=" + this.getId() +
                ", row=" + this.getStartRow() +
                ", col=" + this.getStartCol() +
                ", length=" + this.getLength() +
                ", isPrimary=" + this.isPrimary() +
                ", orientation=" + this.getOrientation() + "]";
    }
}
```

3.2. Board.java

```java
package src.ADT;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 * Class representing the game board for the Rush Hour puzzle.
 */
public class Board {

    // Constants
    public static final int UNKNOWN    = -1;
    public static final int VERTICAL   = 0;
    public static final int HORIZONTAL = 1;
    private static final char EXIT_CHAR = 'K';
    private static final String RESET = "\u001B[0m";

    // Attributes
    private int A;                        // Number of rows in the
    ↪  board
    private int B;                        // Number of columns in
    ↪  the board
    private int N;                        // Number of cars on the
    ↪  board
    private int exitRow;                  // Exit row position
    private int exitCol;                  // Exit column position
    private char[][] grid;                // The game grid
    private List<Car> cars;               // All cars on the board
    private String exitSide;              // Side of the exit
    private String errorMsg;              // Error message for
    ↪  invalid configurations
    private String[] palette;             // Color palette for cars
    private Integer currentMovedCarIndex; // Index of the car that
    ↪  is currently being moved (for highlighting)

    /**
     * Constructor for the Board class
     *
     * @param rows Number of rows in the board
     * @param cols Number of columns in the board
     * @param numCars Number of cars on the board
     * @param exitRow Row position of the exit
     * @param exitCol Column position of the exit
     */
```

```java
public Board(int rows,
             int cols,
             int numCars,
             int exitRow,
             int exitCol,
             String exitSide,
             String errorMsg)
{
    this.A        = rows;
    this.B        = cols;
    this.N        = numCars;
    this.exitRow  = exitRow;
    this.exitCol  = exitCol;
    this.exitSide = exitSide;
    this.grid     = new char[rows][cols];
    this.cars     = new ArrayList<>();
    this.errorMsg = errorMsg;
    this.palette  = generatePalette();
    this.currentMovedCarIndex = null;

    // Initialize the grid with empty cells
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            grid[i][j] = '.';
}

public int getRows() { return this.A; }
public int getCols() { return this.B; }
public int getNumCars() { return this.N; }
public char[][] getGrid() { return this.grid; }
public char getElement(int i, int j) { return this.grid[i][j]; }
public List<Car> getCars() { return this.cars; }
public int getExitRow() { return this.exitRow; }
public int getExitCol() { return this.exitCol; }
public String getExitSide() { return this.exitSide; }
public String getErrorMsg() { return this.errorMsg; }
public boolean hasError() { return this.errorMsg != null; }
public String[] getPalette() { return this.palette; }
public Integer getCurrentMovedCarIndex() { return
↪  this.currentMovedCarIndex; }
public void setCurrentMovedCarIndex(Integer index) {
↪  this.currentMovedCarIndex = index; }

/**
 * Creates a deep copy of the current board state
 *
 * @return A new Board object with the same state
 */
public Board copy()
{
```

```java
        Board newBoard = new Board(this.getRows(),
                                   this.getCols(),
                                   this.getNumCars(),
                                   this.getExitRow(),
                                   this.getExitCol(),
                                   this.getExitSide(),
                                   this.getErrorMsg());

        // Copy the grid
        for (int i = 0; i < this.getRows(); i++)
            System.arraycopy(this.grid[i], 0, newBoard.grid[i], 0,
            ↪  this.getCols());

        // Copy the cars
        for (Car car : this.getCars())
        {
            Car newCar = car.copy();
            newBoard.cars.add(newCar);
        }

        newBoard.exitRow  = this.getExitRow();
        newBoard.exitCol  = this.getExitCol();
        newBoard.errorMsg = this.getErrorMsg();
        newBoard.currentMovedCarIndex = this.currentMovedCarIndex;

        return newBoard;
    }

    /**
     * Check if the board is in a solved state (primary car at exit)
     *
     * @return True if solved, false otherwise
     */
    public boolean isSolved()
    {
        Car primaryCar = getPrimaryCar();
        if (primaryCar == null) return false;

        int startRow = primaryCar.getStartRow();
        int startCol = primaryCar.getStartCol();
        int endRow = startRow + (primaryCar.getOrientation() ==
        ↪  VERTICAL ? primaryCar.getLength() - 1 : 0);
        int endCol = startCol + (primaryCar.getOrientation() ==
        ↪  HORIZONTAL ? primaryCar.getLength() - 1 : 0);

        if (primaryCar.getOrientation() == HORIZONTAL)
        {
            if (exitCol == this.getCols() - 1)
                return (endCol == this.getCols() - 1) && (startRow
                ↪  == exitRow);
```

```java
        else if (exitCol == 0)
            return (startCol == 0) && (startRow == exitRow);
    }
    else
    {
        if (exitRow == this.getRows()-1)
            return (endRow == this.getRows()-1) && (startCol ==
            ↪  exitCol);

        else if (exitRow == 0)
            return (startRow == 0) && (startCol == exitCol);
    }

    return false;
}

/**
 * Get all valid moves for a specific car
 *
 * @param carIndex Index of the car in the cars list
 * @return List of valid move amounts (positive for right/down,
 ↪  negative for left/up)
 */
public List<Integer> getValidMoves(int carIndex)
{
    Car car = cars.get(carIndex);
    List<Integer> validMoves = new ArrayList<>();

    if (car.getOrientation() == HORIZONTAL)
    {
        // Check moves to the left
        int leftMost = car.getStartCol();
        int leftBound = 0;

        for (int col = leftMost - 1; col >= 0; col--)
        {
            if (isOccupied(car.getStartRow(), col))
            {
                leftBound = col + 1;
                break;
            }
        }

        // Add all possible left moves
        for (int col = leftMost - 1; col >= leftBound; col--)
            validMoves.add(col - leftMost); // This will be
            ↪  negative (left)

        // Check moves to the right
```

```java
            int rightMost = car.getStartCol() + car.getLength() - 1;
            int rightBound = this.getCols() - 1;

            for (int col = rightMost + 1; col < this.getCols();
            ↪  col++)
            {
                if (isOccupied(car.getStartRow(), col))
                {
                    rightBound = col - 1;
                    break;
                }
            }

            // Add all possible right moves
            for (int delta = 1; delta <= rightBound - rightMost;
            ↪  delta++)
                validMoves.add(delta); // This will be positive
                ↪  (right)

        }
        else
        {
            // Check moves up
            int topMost = car.getStartRow();
            int topBound = 0;

            for (int row = topMost - 1; row >= 0; row--)
            {
                if (isOccupied(row, car.getStartCol()))
                {
                    topBound = row + 1;
                    break;
                }
            }

            // Add all possible up moves
            for (int row = topMost - 1; row >= topBound; row--)
                validMoves.add(row - topMost); // This will be
                ↪  negative (up)

            // Check moves down
            int bottomMost = car.getStartRow() + car.getLength() -
            ↪  1;
            int bottomBound = this.getRows() - 1;

            for (int row = bottomMost + 1; row < this.getRows();
            ↪  row++)
            {
                if (isOccupied(row, car.getStartCol()))
                {
```

```java
                    bottomBound = row - 1;
                    break;
                }
            }

            // Add all possible down moves
            for (int delta = 1; delta <= bottomBound - bottomMost;
            ↪  delta++)
                validMoves.add(delta); // This will be positive
                ↪  (down)
        }

        return validMoves;
    }

    /**
     * Check if a cell is occupied by any car
     *
     * @param row Row index
     * @param col Column index
     * @return True if occupied, false otherwise
     */
    private boolean isOccupied(int row, int col)
    {
        // Check if cell is within bounds
        if (row < 0 || row >= this.getRows() || col < 0 || col >=
        ↪  this.getCols())
            return true; // Treat out-of-bounds as occupied

        return grid[row][col] != '.';
    }

    /**
     * Apply a move to a car
     *
     * @param carIndex Index of the car in the cars list
     * @param move Amount to move (positive for right/down, negative
     ↪  for left/up)
     * @return A new Board with the move applied
     */
    public Board applyMove(int carIndex, int move)
    {
        Board newBoard = this.copy();
        Car car = newBoard.cars.get(carIndex);

        // Clear the current car position
        int[][] oldCells = car.getOccupiedCells();
        for (int[] cell : oldCells)
            newBoard.grid[cell[0]][cell[1]] = '.';
```

```java
        // Move the car
        car.move(move);

        // Update the grid with the new car position
        int[][] newCells = car.getOccupiedCells();
        for (int[] cell : newCells)
            newBoard.grid[cell[0]][cell[1]] = car.getId();

        newBoard.setCurrentMovedCarIndex(carIndex);

        return newBoard;
    }

    /**
     * Get a string representation of the current board state with
     * colored cars
     *
     * @return String representation of the board
     */
    @Override
    public String toString()
    {
        StringBuilder sb = new StringBuilder();

        if (exitSide != null && exitSide.equalsIgnoreCase("TOP"))
        {
            // Case: Top exit
            for (int j = 0; j < this.getCols(); j++)
            {
                if (j == exitCol)
                {
                    int exitColorIndex = EXIT_CHAR - 'A';
                    if (exitColorIndex >= 0 && exitColorIndex <
                        palette.length)
                    sb.append(palette[exitColorIndex])
                        .append(EXIT_CHAR).append(RESET);

                    else sb.append(EXIT_CHAR);
                }
                else sb.append(' ');
            }
            sb.append("\n");
        }

        for (int i = 0; i < this.getRows(); i++)
        {
            // Case: Left exit
            if (exitSide != null &&
                exitSide.equalsIgnoreCase("LEFT") &&
                i == exitRow)
```

```
{
    int exitColorIndex = EXIT_CHAR - 'A';
    if (exitColorIndex >= 0 && exitColorIndex <
        palette.length)

    sb.append(palette[exitColorIndex])
        .append(EXIT_CHAR).append(RESET);

    else sb.append(EXIT_CHAR);
}
else if (exitSide != null &&
↪   exitSide.equalsIgnoreCase("LEFT"))
    sb.append(' ');

// Add the main grid
for (int j = 0; j < this.getCols(); j++)
{
    char cell = grid[i][j];

    if (cell == '.') sb.append(cell);
    else
    {
        boolean shouldColor =
        (currentMovedCarIndex == null) ||
        (currentMovedCarIndex != null &&
        cars.get(currentMovedCarIndex).getId() == cell);

        if (shouldColor)
        {
            int colorIndex = cell - 'A';
            if (colorIndex >= 0 && colorIndex <
                palette.length)
            sb.append(palette[colorIndex])
                .append(cell).append(RESET);

            else sb.append(cell);
        }
        else sb.append(cell);
    }
}

// Case: Right exit
if (exitSide != null &&
↪   exitSide.equalsIgnoreCase("RIGHT") && i == exitRow)
{
    int exitColorIndex = EXIT_CHAR - 'A';
    if (exitColorIndex >= 0 && exitColorIndex <
        palette.length)
    sb.append(palette[exitColorIndex])
        .append(EXIT_CHAR).append(RESET);
```

```java
                else sb.append(EXIT_CHAR);
            }
            else if (exitSide != null &&
            ↪   exitSide.equalsIgnoreCase("RIGHT"))
                sb.append(' ');

            if (i < this.getRows() - 1) sb.append("\n");
        }

        // Case: Bottom exit
        if (exitSide != null && exitSide.equalsIgnoreCase("BOTTOM"))
        {
            sb.append("\n");
            for (int j = 0; j < this.getCols(); j++)
            {
                if (j == exitCol)
                {
                    int exitColorIndex = EXIT_CHAR - 'A';
                    if (exitColorIndex >= 0 && exitColorIndex <
                        palette.length)
                    sb.append(palette[exitColorIndex])
                        .append(EXIT_CHAR).append(RESET);

                    else sb.append(EXIT_CHAR);
                }
                else sb.append(' ');
            }
        }

        return sb.toString();
    }

    /**
     * Get a string representation of the cars on the board
     *
     * @return String representation of the cars
     */
    @Override
    public int hashCode()
    {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < this.getRows(); i++)
            for (int j = 0; j < this.getCols(); j++)
                sb.append(grid[i][j]);

        return sb.toString().hashCode();
    }

    /**
```

```java
     * Check if two boards are equal
     *
     * @param obj The object to compare with
     * @return True if equal, false otherwise
     */
    @Override
    public boolean equals(Object obj)
    {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return
        ↪  false;

        Board other = (Board) obj;
        if (this.getRows() != other.getRows() || this.getCols() !=
        ↪  other.getCols()) return false;

        for (int i = 0; i < this.getRows(); i++)
            for (int j = 0; j < this.getCols(); j++)
                if (grid[i][j] != other.grid[i][j])
                    return false;

        return true;
    }

    /**
     * Get Primary Car
     *
     * @return Primary Car
     */
    public Car getPrimaryCar()
    {
        for (Car car : this.getCars())
        {
            if (car.isPrimary())
                return car;
        }
        return null;
    }

    /**
     * Generates a palette of colors for the cars.
     *
     * @return Array of ANSI color codes
     */
    public String[] generatePalette()
    {
        String[] palette =
        {
            "\u001B[38;2;50;255;50m",    // A: Bright Green
            ↪  (#32FF32)
```

```
            "\u001B[38;2;200;100;255m",  // B: Light Purple
            ↪  (#C864FF)
            "\u001B[38;2;255;215;0m",    // C: Gold (#FFD700)
            "\u001B[38;2;255;255;50m",   // D: Bright Yellow
            ↪  (#FFFF32)
            "\u001B[38;2;255;50;255m",   // E: Bright Magenta
            ↪  (#FF32FF)
            "\u001B[38;2;50;255;255m",   // F: Bright Cyan (#32FFFF)
            "\u001B[38;2;255;128;0m",    // G: Orange (#FF8000)
            "\u001B[38;2;128;255;128m",  // H: Light Green (#80FF80)
            "\u001B[38;2;180;180;255m",  // I: Light Blue (#B4B4FF)
            "\u001B[38;2;255;200;100m",  // J: Light Orange
            ↪  (#FFC864)
            "\u001B[38;2;50;150;255m",   // K: Bright Blue (#3296FF)
            "\u001B[38;2;100;255;200m",  // L: Light Turquoise
            ↪  (#64FFC8)
            "\u001B[38;2;255;150;150m",  // M: Light Red (#FF9696)
            "\u001B[38;2;150;255;150m",  // N: Pale Green (#96FF96)
            "\u001B[38;2;255;160;122m",  // O: Light Salmon
            ↪  (#FFA07A)
            "\u001B[38;2;255;50;50m",    // P: Bright Red (#FF3232)
            "\u001B[38;2;173;255;47m",   // Q: Green Yellow
            ↪  (#ADFF2F)
            "\u001B[38;2;64;224;208m",   // R: Turquoise (#40E0D0)
            "\u001B[38;2;238;130;238m",  // S: Violet (#EE82EE)
            "\u001B[38;2;255;160;160m",  // T: Light Coral (#FFA0A0)
            "\u001B[38;2;135;206;250m",  // U: Light Sky Blue
            ↪  (#87CEFA)
            "\u001B[38;2;200;200;200m",  // V: Light Gray (#C8C8C8)
            "\u001B[38;2;255;100;100m",  // W: Salmon (#FF6464)
            "\u001B[38;2;255;105;180m",  // X: Hot Pink (#FF69B4)
            "\u001B[38;2;150;255;0m",    // Y: Bright Lime (#96FF00)
            "\u001B[38;2;255;80;80m"     // Z: Light Crimson
            ↪  (#FF5050)
        };
        return palette;
    }
}
```

3.3. State.java

```java
package src.ADT;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;

public class State implements Comparable<State>
{
    public static final int INVALID = 69420;
    private Board board;
    private List<int[]> moves; // History of moves: [carIndex,
    ↪  moveAmount, pieceId]
    private int gValue;        // Cost so far: for UCS and A*
    private int hValue;        // Heuristic value: for A* and Greedy
    private int fValue;        // For A*: f(n) = g(n) + h(n)

    /**
     * Constructor for UCS
     *
     * @param board The current board state
     * @param moves The list of moves made to reach this state
     */
    public State(Board board, List<int[]> moves)
    {
        this.board = board;
        this.moves = new ArrayList<>(moves);
        calculateGValue();
    }

    /**
     * Constructor for Greedy and A*
     *
     * @param board The current board state
     * @param moves The list of moves made to reach this state
     * @param hValue The heuristic value for this state
     */
    public State(Board board, List<int[]> moves, int hValue)
    {
        this.board = board;
        this.moves = new ArrayList<>(moves);
        calculateGValue();
        this.hValue = hValue;
        calculateFValue();
    }

    public Board getBoard() { return board; }
```

```java
public List<int[]> getMoves() { return moves; }
public int getGValue() { return gValue; }
public int getHValue() { return hValue; }
public int getFValue() { return fValue; }

/**
 * Convert state to String for visitedStates
 *
 * @param board The current board state
 * @return A string representation of the board state
 */
public static String getBoardStateString(Board board)
{
    StringBuilder sb = new StringBuilder();
    for (Car car : board.getCars())
    {
        sb.append(car.getId()).append(car.getStartRow())
        .append(car.getStartCol());
    }
    return sb.toString();
}

/**
 * Calculate the g(n) value for UCS
 */
public void calculateGValue() { this.gValue = this.moves.size();
↪  }

/**
 * Calculate the f(n) = g(n) + h(n) value for A*
 */
public void calculateFValue() { this.fValue = getGValue() +
↪  getHValue(); }

/**
 * Calculate the heuristic value for the blocking car heuristic
 *
 * @param board
 * @return The heuristic value for the blocking car heuristic
 */
public static int calculateBlockingCarHeuristic(Board board)
{
    Car primaryCar = board.getPrimaryCar();

    if (primaryCar == null) return INVALID;

    int exitRow = board.getExitRow();
    int exitCol = board.getExitCol();
    int blockingCars = 0;
```

```java
        if (primaryCar.getOrientation() == 1)
        {
            int leftEdge = primaryCar.getStartCol();
            int rightEdge = primaryCar.getStartCol() +
            ↪  primaryCar.getLength() - 1;

            if (exitCol == board.getCols())
            {
                distance = exitCol - rightEdge - 1;

                Set<Character> blockingPieceIds = new HashSet<>();
                for (int col = rightEdge + 1; col < exitCol; col++)
                {
                    if (col >= 0 &&
                        col < board.getCols() &&
                        primaryCar.getStartRow() >= 0 &&
                        primaryCar.getStartRow() < board.getRows())
                    {
                        char cell =
                        ↪  board.getGrid()[primaryCar.getRow()][col];
                        if (cell != '.') blockingPieceIds.add(cell);
                    }
                }
                blockingCars = blockingPieceIds.size();
            }
            else if (exitCol == 0)
            {
                Set<Character> blockingPieceIds = new HashSet<>();
                for (int col = leftEdge - 1; col > exitCol; col--)
                {
                    if (col >= 0 &&
                        col < board.getCols() &&
                        primaryCar.getStartRow() >= 0 &&
                        primaryCar.getStartRow() < board.getRows())
                    {
                        char cell =
                        ↪  board.getGrid()[primaryCar.getRow()][col];
                        if (cell != '.') blockingPieceIds.add(cell);
                    }
                }
                blockingCars = blockingPieceIds.size();
            }
        }
        else
        {
            int topEdge = primaryCar.getStartRow();
            int bottomEdge = primaryCar.getStartRow() +
            ↪  primaryCar.getLength() - 1;

            if (exitRow == board.getRows())
```

```java
            {
                Set<Character> blockingPieceIds = new HashSet<>();
                for (int row = bottomEdge + 1; row < exitRow; row++)
                {
                    if (row >= 0 && row < board.getRows() &&
                    ↪  primaryCar.getStartCol() >= 0 &&
                    ↪  primaryCar.getStartCol() < board.getCols())
                    ↪  {
                        char cell =
                        ↪  board.getGrid()[row][primaryCar.getCol()];
                        if (cell != '.') blockingPieceIds.add(cell);
                    }
                }
                blockingCars = blockingPieceIds.size();
            }
            else if (exitRow == 0)
            {
                Set<Character> blockingPieceIds = new HashSet<>();
                for (int row = topEdge - 1; row > exitRow; row--)
                {
                    if (row >= 0 && row < board.getRows() &&
                        primaryCar.getStartCol() >= 0 &&
                        primaryCar.getStartCol() < board.getCols())
                    {
                        char cell =
                        board.getGrid()[row][primaryCar.getCol()];
                        if (cell != '.') blockingPieceIds.add(cell);
                    }
                }
                blockingCars = blockingPieceIds.size();
            }
        }

        return blockingCars;
    }

    /**
     * Calculate the heuristic value for the Manhattan distance
     ↪  heuristic
     *
     * @param board
     * @return The heuristic value for the Manhattan distance
     ↪  heuristic
     */
    public static int calculateManhattanDistanceHeuristic(Board
    ↪  board)
    {
        Car primaryCar = board.getPrimaryCar();

        if (primaryCar == null) return INVALID;
```

```java
        int exitRow = board.getExitRow();
        int exitCol = board.getExitCol();

        if (primaryCar.getOrientation() == 1)
        {
            int leftEdge = primaryCar.getStartCol();
            int rightEdge = primaryCar.getStartCol() +
            ↪  primaryCar.getLength() - 1;

            if (exitCol == board.getCols())
                return Math.max(0, exitCol - rightEdge - 1);

            else if (exitCol == -1)
                return Math.max(0, leftEdge - (exitCol + 1));
        }
        else
        {
            int topEdge = primaryCar.getStartRow();
            int bottomEdge = primaryCar.getStartRow() +
            ↪  primaryCar.getLength() - 1;

            if (exitRow == board.getRows())
                return Math.max(0, exitRow - bottomEdge - 1);

            else if (exitRow == -1)
                return Math.max(0, topEdge - (exitRow + 1));
        }

        return 0;
    }

    @Override
    public int compareTo(State other) { return
    ↪  Integer.compare(this.fValue, other.fValue); }

    @Override
    public int hashCode() { return Objects.hash(board); }

    @Override
    public boolean equals(Object o)
    {
        if (this == o)
            return true;

        if (o == null || getClass() != o.getClass())
            return false;

        State state = (State) o;
        return Objects.equals(board, state.board);
```

```java
    }

    /**
     * Add a move to the history
     * @param carIndex The index of the car moved
     * @param moveAmount The amount moved (positive or negative)
     * @param newBoard The new board state after the move
     * @param newHValue The heuristic value for the new state
     * @return A new State object with the updated board and moves
     */
    public State addMove(int carIndex, int moveAmount, Board
    ↪  newBoard, int newHValue)
    {
        List<int[]> newMoves = new ArrayList<>(moves);
        char carId = board.getCars().get(carIndex).getId();
        newMoves.add(new int[]{carIndex, moveAmount, (int)carId});
        return new State(newBoard, newMoves, newHValue);
    }

    /**
     * Get a string representation of the moves made
     *
     * @return A string describing the moves made
     */
    public String getMovesAsString()
    {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < moves.size(); i++)
        {
            int[] move = moves.get(i);
            int carIndex = move[0];
            int moveAmount = move[1];
            char pieceId = (char)move[2];

            String direction = "";
            if (moveAmount > 0)
                direction = " right/down " + moveAmount + " steps";

            else
                direction = " left/up " + Math.abs(moveAmount) + "
                ↪  steps";

            sb.append("Move ")
              .append(i + 1)
              .append(": Piece '")
              .append(pieceId)
              .append("' (index ")
              .append(carIndex)
              .append(")")
              .append(direction);
```

```java
            if (i < moves.size() - 1) sb.append("\n");
        }
        return sb.toString();
    }

    /**
     * Get the last moved piece ID
     *
     * @return The ID of the last moved piece, or '\0' if no moves
     ↪  were made
     */
    public char getLastMovedPieceId()
    {
        if (moves.isEmpty())
            return '\0';

        int[] lastMove = moves.get(moves.size() - 1);
        return (char)lastMove[2];
    }

    /**
     * Get the last move description
     *
     * @return A string describing the last move made
     */
    public String getLastMoveDescription()
    {
        if (moves.isEmpty()) return "No moves made";
        int[] lastMove = moves.get(moves.size() - 1);
        int moveAmount = lastMove[1];
        char pieceId = (char)lastMove[2];
        String direction = moveAmount > 0 ? "right/down" :
        ↪  "left/up";
        return "Piece '" + pieceId + "' moved " + direction + " " +
        ↪  Math.abs(moveAmount) + " steps";
    }
}
```

3.4. Algorithm.java

```java
package src.Algorithm;

import src.ADT.*;
import java.util.*;

public abstract class Algorithm
{
    protected Board initialBoard;
    protected Set<String> visitedStates;
    protected int nodesExplored;
    protected long executionTime;

    /**
     * Constructor for the Algorithm class
     *
     * @param board The initial board state
     */
    public Algorithm(Board board)
    {
        this.initialBoard = board;
        this.visitedStates = new HashSet<>();
        this.nodesExplored = 0;
        this.executionTime = 0;
    }

    public int getNodesExplored() { return nodesExplored; }
    public long getExecutionTime() { return executionTime; }
    protected void setExecutionTime(long time) { this.executionTime
    ↪  = time; }
    protected long startTiming() { return
    ↪  System.currentTimeMillis(); }
    protected void endTiming(long startTime)
    {
        this.executionTime = System.currentTimeMillis() - startTime;
    }

    /**
     * Solve the puzzle using the appropriate algorithm
     *
     * @param heuristic The heuristic to use (if applicable)
     * @return A list of moves to solve the puzzle
     */
    public abstract List<int[]> solve(String heuristic);

    /**
     * Split multi-unit moves into individual steps
     *
     * @param moves The list of moves to split
```

```java
     * @return A list of single-step moves
     */
    public static List<int[]> splitMovesToSteps(List<int[]> moves)
    {
        List<int[]> steppedMoves = new ArrayList<>();

        for (int[] move : moves)
        {
            int pieceIndex = move[0];
            int moveAmount = move[1];

            // Split the move into individual steps of size 1
            if (moveAmount > 0)
            {
                // Positive movement: add moveAmount number of
                ↪  [pieceIndex, 1] moves
                for (int i = 0; i < moveAmount; i++)
                    steppedMoves.add(new int[]{pieceIndex, 1});
            }
            else if (moveAmount < 0)
            {
                // Negative movement: add |moveAmount| number of
                ↪  [pieceIndex, -1] moves
                for (int i = 0; i < Math.abs(moveAmount); i++)
                    steppedMoves.add(new int[]{pieceIndex, -1});
            }
            // If moveAmount is 0, we don't add anything (no
            ↪  movement)
        }

        return steppedMoves;
    }

    /**
     * Helper method to add a state to the visited set
     *
     * @param board The board state to add
     */
    protected void addToVisited(Board board)
    {
        String boardStr = State.getBoardStateString(board);
        visitedStates.add(boardStr);
    }

    /**
     * Helper method to check if a state has been visited
     *
     * @param board The board state to check
     * @return true if the state has been visited, false otherwise
     */
```

```java
    protected boolean hasBeenVisited(Board board)
    {
        String boardStr = State.getBoardStateString(board);
        return visitedStates.contains(boardStr);
    }

    /**
     * Increment the number of nodes explored
     */
    protected void incrementNodesExplored() { nodesExplored++; }
}
```

3.4.1 UCS.java

```java
package src.Algorithm;
import src.ADT.*;

import java.util.*;

public class UCS extends Algorithm
{
    public UCS(Board board)
    {
        super(board);
    }

    @Override
    public List<int[]> solve(String heuristic)
    {
        // UCS doesn't use heuristics, ignore the parameter
        return solveUCS();
    }

    private List<int[]> solveUCS()
    {
        long startTime = startTiming();

        PriorityQueue<State> queue = new
        ↪  PriorityQueue<>(Comparator.comparingInt(s ->
        ↪  s.getGValue()));
        List<int[]> initialMoves = new ArrayList<>();

        // UCS only uses g(n) - the cost from start to current node
        queue.add(new State(initialBoard, initialMoves));

        while (!queue.isEmpty())
        {
            State curState = queue.poll();
            Board curBoard = curState.getBoard();
            List<Car> pieces = curBoard.getCars();
            List<int[]> curMoves = curState.getMoves();
            incrementNodesExplored();
```

```java
            if (curBoard.isSolved())
            {
                endTiming(startTime);
                return splitMovesToSteps(curMoves);
            }

            if (hasBeenVisited(curBoard)) continue;
            addToVisited(curBoard);

            for (int i = 0; i < pieces.size(); i++)
            {
                List<Integer> validMoves =
                ↪  curBoard.getValidMoves(i);
                for(Integer moveAmount : validMoves)
                {
                    Board newBoard = curBoard.copy();
                    newBoard = newBoard.applyMove(i, moveAmount);

                    if (!hasBeenVisited(newBoard))
                    {
                        List<int[]> newMoves = new
                        ↪  ArrayList<>(curMoves);
                        newMoves.add(new int[]{i, moveAmount});

                        // UCS: Create new state with updated moves
                        queue.add(new State(newBoard, newMoves));
                    }
                }
            }
        }

        endTiming(startTime);
        return new ArrayList<>();
    }
}
```

3.4.2 GBFS.java

```java
package src.Algorithm;

import src.ADT.*;
import java.util.*;

public class GBFS extends Algorithm
{
    public GBFS(Board board)
    {
        super(board);
    }

    @Override
    public List<int[]> solve(String heuristic)
    {
```

```java
        if(heuristic.equals("Blocking")) return solveBlockingCars();
        else if(heuristic.equals("Manhattan")) return solveManhattanDistance();
        else throw new IllegalArgumentException("Invalid heuristic: " + heuristic);
    }

    private List<int[]> solveBlockingCars()
    {
        long startTime = startTiming();

        PriorityQueue<State> queue = new PriorityQueue<>(Comparator.comparingInt(s -> s.getHValue()));
        List<int[]> initialMoves = new ArrayList<>();

        int initialHValue = State.calculateBlockingCarHeuristic(initialBoard);
        queue.add(new State(initialBoard, initialMoves, initialHValue));

        while (!queue.isEmpty())
        {
            State curState = queue.poll();
            Board curBoard = curState.getBoard();
            List<Car> cars = curBoard.getCars();
            List<int[]> curMoves = curState.getMoves();
            incrementNodesExplored();

            if (curBoard.isSolved())
            {
                endTiming(startTime);
                return splitMovesToSteps(curMoves);
            }

            if (hasBeenVisited(curBoard)) continue;
            addToVisited(curBoard);

            for (int i = 0; i < cars.size(); i++)
            {
                List<Integer> validMoves = curBoard.getValidMoves(i);
                for(Integer moveAmount : validMoves)
                {
                    Board newBoard = curBoard.copy();
                    newBoard = newBoard.applyMove(i, moveAmount);

                    if (!hasBeenVisited(newBoard))
                    {
                        List<int[]> newMoves = new ArrayList<>(curMoves);
                        newMoves.add(new int[]{i, moveAmount});

                        int hValue = State.calculateBlockingCarHeuristic(newBoard);
```

```
                    queue.add(new State(newBoard, newMoves,
                    ↪  hValue));
                }
            }
        }
    }

    endTiming(startTime);
    return new ArrayList<>();
}

private List<int[]> solveManhattanDistance()
{
    long startTime = startTiming();

    PriorityQueue<State> queue = new
    ↪  PriorityQueue<>(Comparator.comparingInt(s ->
    ↪  s.getHValue()));
    List<int[]> initialMoves = new ArrayList<>();

    int initialHValue =
    ↪  State.calculateManhattanDistanceHeuristic(initialBoard);
    queue.add(new State(initialBoard, initialMoves,
    ↪  initialHValue));

    while (!queue.isEmpty())
    {
        State curState = queue.poll();
        Board curBoard = curState.getBoard();
        List<Car> cars = curBoard.getCars();
        List<int[]> curMoves = curState.getMoves();
        incrementNodesExplored();

        if (curBoard.isSolved())
        {
            endTiming(startTime);
            return curMoves;
        }

        if(hasBeenVisited(curBoard)) continue;
        addToVisited(curBoard);

        for (int i = 0; i < cars.size(); i++)
        {
            List<Integer> validMoves =
            ↪  curBoard.getValidMoves(i);
            for (Integer moveAmount : validMoves)
            {
                Board newBoard = curBoard.copy();
                newBoard = newBoard.applyMove(i, moveAmount);

                if (!hasBeenVisited(newBoard))
                {
                    List<int[]> newMoves = new
                    ↪  ArrayList<>(curMoves);
                    newMoves.add(new int[]{i, moveAmount});
```

```
                        int hValue =
                        ↪  State.calculateManhattanDistanceHeuristic
                        (newBoard);
                        queue.add(new State(newBoard, newMoves,
                        ↪  hValue));
                    }
                }
            }
        }

        endTiming(startTime);
        return new ArrayList<>();
    }
}
```

3.4.3 AStar.java

```
package src.Algorithm;

import src.ADT.*;
import java.util.*;

public class AStar extends Algorithm
{
    public AStar(Board board)
    {
        super(board);
    }

    @Override
    public List<int[]> solve(String heuristic)
    {
        if (heuristic.equals("Blocking")) return
        ↪  solveBlockingCars();
        else if (heuristic.equals("Manhattan")) return
        ↪  solveManhattanDistance();
        else throw new IllegalArgumentException("Invalid heuristic:
        ↪  " + heuristic);
    }

    private List<int[]> solveBlockingCars()
    {
        long startTime = startTiming();

        PriorityQueue<State> queue = new
        ↪  PriorityQueue<>(Comparator.comparingInt(s ->
        ↪  s.getFValue()));
        List<int[]> initialMoves = new ArrayList<>();

        int intialHValue =
        ↪  State.calculateBlockingCarHeuristic(initialBoard);

        addToVisited(initialBoard);
        queue.add(new State(initialBoard, initialMoves,
        ↪  intialHValue));
```

```java
        while (!queue.isEmpty())
        {
            State curState = queue.poll();
            Board curBoard = curState.getBoard();
            List<Car> pieces = curBoard.getCars();
            List<int[]> curMoves = curState.getMoves();
            incrementNodesExplored();

            if (curBoard.isSolved())
            {
                endTiming(startTime);
                return curMoves;
            }

            // Set maximum cost limit
            int maxCost = curBoard.getRows() * curBoard.getCols() *
            ↪  50;
            if (curMoves.size() > maxCost) continue;

            for (int i = 0; i < pieces.size(); i++)
            {
                List<Integer> validMoves =
                ↪  curBoard.getValidMoves(i);
                for (Integer moveAmount : validMoves)
                {
                    Board newBoard = curBoard.copy();
                    newBoard = newBoard.applyMove(i, moveAmount);

                    if (!hasBeenVisited(newBoard))
                    {
                        addToVisited(newBoard);

                        List<int[]> newMoves = new
                        ↪  ArrayList<>(curMoves);
                        newMoves.add(new int[]{i, moveAmount});

                        int hValue =
                        ↪  State.calculateBlockingCarHeuristic
                        (newBoard);
                        queue.add(new State(newBoard, newMoves,
                        ↪  hValue));
                    }
                }
            }
        }

        endTiming(startTime);
        return new ArrayList<>();
    }

    private List<int[]> solveManhattanDistance()
    {
        long startTime = startTiming();
```

```java
        PriorityQueue<State> queue = new
        ↪  PriorityQueue<>(Comparator.comparingInt(s ->
        ↪  s.getFValue()));
        List<int[]> initialMoves = new ArrayList<>();

        int initialHValue =
        ↪  State.calculateManhattanDistanceHeuristic(initialBoard);

        addToVisited(initialBoard);
        queue.add(new State(initialBoard, initialMoves,
        ↪  initialHValue));

        while (!queue.isEmpty())
        {
            State curState = queue.poll();
            Board curBoard = curState.getBoard();
            List<Car> pieces = curBoard.getCars();
            List<int[]> curMoves = curState.getMoves();

            incrementNodesExplored();

            if (curBoard.isSolved())
            {
                endTiming(startTime);
                return splitMovesToSteps(curMoves);
            }

            int maxCost = curBoard.getRows() * curBoard.getCols() *
            ↪  50;
            if (curMoves.size() > maxCost) continue;

            for (int i = 0; i < pieces.size(); i++)
            {
                List<Integer> validMoves =
                ↪  curBoard.getValidMoves(i);
                for (Integer moveAmount : validMoves)
                {
                    Board newBoard = curBoard.copy();
                    newBoard = newBoard.applyMove(i, moveAmount);

                    // Check if state has been visited BEFORE
                    ↪  adding to queue
                    if (!hasBeenVisited(newBoard))
                    {
                        addToVisited(newBoard);

                        List<int[]> newMoves = new
                        ↪  ArrayList<>(curMoves);
                        newMoves.add(new int[]{i, moveAmount});

                        int hValue =
                        ↪  State.calculateManhattanDistanceHeuristic
                        (newBoard);
                        queue.add(new State(newBoard, newMoves,
                        ↪  hValue));
                    }
                }
```

```
            }
        }

        endTiming(startTime);
        return new ArrayList<>();
    }
}
```

3.4.4 Fringe.java

```
package src.Algorithm;

import src.ADT.*;
import java.util.*;

public class Fringe extends Algorithm
{
    public Fringe(Board board)
    {
        super(board);
    }

    @Override
    public List<int[]> solve(String heuristic)
    {
        if (heuristic.equals("Blocking")) return
        ↪  solveBlockingCars();
        else if (heuristic.equals("Manhattan")) return
        ↪  solveManhattanDistance();
        else throw new IllegalArgumentException("Invalid heuristic:
        ↪  " + heuristic);
    }

    private List<int[]> solveBlockingCars()
    {
        long startTime = startTiming();

        List<State> fringe = new ArrayList<>();
        List<int[]> initialMoves = new ArrayList<>();

        int initialHValue =
        ↪  State.calculateBlockingCarHeuristic(initialBoard);
        int fLimit = initialHValue;

        addToVisited(initialBoard);
        fringe.add(new State(initialBoard, initialMoves,
        ↪  initialHValue));

        while (!fringe.isEmpty())
        {
            Collections.sort(fringe, Comparator.comparingInt(s ->
            ↪  s.getFValue()));

            int nextFLimit = Integer.MAX_VALUE;
            List<State> nextFringe = new ArrayList<>();
```

```java
for (State curState : fringe) {
    Board curBoard = curState.getBoard();
    List<Car> pieces = curBoard.getCars();
    List<int[]> curMoves = curState.getMoves();
    int curFValue = curState.getFValue();

    if (curFValue > fLimit) {
        nextFLimit = Math.min(nextFLimit, curFValue);
        nextFringe.add(curState);
        continue;
    }

    incrementNodesExplored();

    if (curBoard.isSolved())
    {
        endTiming(startTime);
        return curMoves;
    }

    int maxCost = curBoard.getRows() *
    ↪  curBoard.getCols() * 50;
    if (curMoves.size() > maxCost) continue;

    for (int i = 0; i < pieces.size(); i++)
    {
        List<Integer> validMoves =
        ↪  curBoard.getValidMoves(i);
        for (Integer moveAmount : validMoves)
        {
            Board newBoard = curBoard.copy();
            newBoard = newBoard.applyMove(i,
            ↪  moveAmount);

            if (!hasBeenVisited(newBoard))
            {
                addToVisited(newBoard);

                List<int[]> newMoves = new
                ↪  ArrayList<>(curMoves);
                newMoves.add(new int[]{i, moveAmount});

                int hValue =
                ↪  State.calculateBlockingCarHeuristic
                (newBoard);
                int fValue = newMoves.size() + hValue;

                State newState = new State(newBoard,
                ↪  newMoves, hValue);

                if (fValue <= fLimit) {
                    nextFringe.add(newState);
                } else {
                    nextFLimit = Math.min(nextFLimit,
                    ↪  fValue);
                    nextFringe.add(newState);
```

```
                        }
                    }
                }
            }
        }

        if (nextFLimit == Integer.MAX_VALUE &&
        ↪  nextFringe.isEmpty()) {
            endTiming(startTime);
            return new ArrayList<>();
        }
        fLimit = nextFLimit;
        fringe = nextFringe;
    }
    endTiming(startTime);
    return new ArrayList<>();
}

private List<int[]> solveManhattanDistance()
{
    long startTime = startTiming();

    List<State> fringe = new ArrayList<>();
    List<int[]> initialMoves = new ArrayList<>();

    int initialHValue =
    ↪  State.calculateManhattanDistanceHeuristic(initialBoard);
    int fLimit = initialHValue;

    addToVisited(initialBoard);
    fringe.add(new State(initialBoard, initialMoves,
    ↪  initialHValue));

    while (!fringe.isEmpty())
    {
        Collections.sort(fringe, Comparator.comparingInt(s ->
        ↪  s.getFValue()));

        int nextFLimit = Integer.MAX_VALUE;
        List<State> nextFringe = new ArrayList<>();

        for (State curState : fringe) {
            Board curBoard = curState.getBoard();
            List<Car> pieces = curBoard.getCars();
            List<int[]> curMoves = curState.getMoves();
            int curFValue = curState.getFValue();

            if (curFValue > fLimit) {
                nextFLimit = Math.min(nextFLimit, curFValue);
                nextFringe.add(curState);
                continue;
            }
            incrementNodesExplored();

            if (curBoard.isSolved())
            {
                endTiming(startTime);
```

```
                    return splitMovesToSteps(curMoves);
                }

                int maxCost = curBoard.getRows() *
                ↪  curBoard.getCols() * 50;
                if (curMoves.size() > maxCost) continue;

                for (int i = 0; i < pieces.size(); i++)
                {
                    List<Integer> validMoves =
                    ↪  curBoard.getValidMoves(i);
                    for (Integer moveAmount : validMoves)
                    {
                        Board newBoard = curBoard.copy();
                        newBoard = newBoard.applyMove(i,
                        ↪  moveAmount);

                        if (!hasBeenVisited(newBoard))
                        {
                            addToVisited(newBoard);

                            List<int[]> newMoves = new
                            ↪  ArrayList<>(curMoves);
                            newMoves.add(new int[]{i, moveAmount});

                            int hValue =
                            ↪  State.calculateManhattanDistanceHeuristic
                            (newBoard);
                            int fValue = newMoves.size() + hValue;

                            State newState = new State(newBoard,
                            ↪  newMoves, hValue);

                            if (fValue <= fLimit) {
                                nextFringe.add(newState);
                            } else {
                                nextFLimit = Math.min(nextFLimit,
                                ↪  fValue);
                                nextFringe.add(newState);
                            }
                        }
                    }
                }
            }

            if (nextFLimit == Integer.MAX_VALUE &&
            ↪  nextFringe.isEmpty()) {
                endTiming(startTime);
                return new ArrayList<>();
            }
            fLimit = nextFLimit;
            fringe = nextFringe;
        }
        endTiming(startTime);
        return new ArrayList<>();
    }
}
```

3.5. Penjelasan Bonus yang diimplementasikan

Secara garis besar, seluruh bonus yang ada telah dikerjakan pada tugas kecil ini. Berikut penjelasan terkait masing-masing implementasinya.

3.5.1 Algoritma Alternatif

Untuk bonus algoritma alternatif, diimplementasikan algoritma *Fringe Search* yang telah dijelaskan pada Bab 2.5 dengan lengkap.

Secara singkat, *Fringe Search* menggabungkan kelebihan dari *A* Search* dan *Iterative Deepening Search*, dimana untuk mengatasi *memory overhead* yang acapkali dialami oleh *A* Search* diatasi dengan menggunakan *threshold* dari *f-value*. *F-value* akan diinkrementasi secara bertahap sesuai dengan nilai *f-value* yang dieksplorasi dimana diambil digunakan *f-value* terkecil yang lebih besar dari *threshold*. Inkrementasi secara bertahap ini layaknya *Iterative Deepening Search*.

3.5.2 Heuristik Alternatif

Untuk bonus algoritma alternatif, telah dijelaskan kedua heuristik secara lengkap pada Bab 2.6.

Dalam implementasinya, kedua heuristik ini terdapat pada `State.java`. Cara kedua heuristik diimplementasikan sama persis dengan definisinya. Heuristic `Manhattan Distance` mengambil terlebih dahulu posisi pintu keluar dan kemudian mengambil *cell* dari *primary piece* yang terdekat dengan pintu keluar. Setelah itu, melakukan kalkulasi *distance* sesuai persamaan `Manhattan Distance`, sedangkan heuristic `Blocking Cars` mengambil posisi pintu keluar dan kemudian menghitung berapa banyak *cell* yang *occupied* hingga bertemu dengan ujung *primary piece* yang terdekat dengan pintu keluar.

3.5.3 *Graphical User Interface*

Bonus GUI diimplementasikan menggunakan *library* JavaFX pada dua komponen utama, yaitu `GUI.java` dan `Animation.java`. Pada awal program dijalankan, GUI akan memanggil fungsi `start` dan menampilkan *sidebar* dan *main layout*.

Pada *sidebar*, terdapat tombol `Load .txt` untuk memuat konfigurasi yang ingin dicari solusinya. Setelah melakukan *load*, barulah tombol `Algorithm` akan diaktifkan dan dapat dipilih. Kemudian, memilih algoritma dan jika algoritma tersebut memakai heuristik atau $h(n)$, perlu memilih heuristik terlebih dahulu sebelum dapat menekan tombol `Solve!` untuk mencari solusi dari konfigurasi yang diberikan.

Pada *Main Layout*, terdapat *speed slider*, tombol *play*, tombol *pause*, dan tombol *stop* untuk memudahkan interaksi bagaimana pengguna ingin melihat animasinya. Apabila pengguna tidak ingin menunggu waktu yang cukup lama dengan solusi yang menghasilkan *moves* yang banyak, cukup atur kecepatan animasi menjadi maksimal pada *slider*.

Adapun jika pengguna ingin mengulang animasi di tengah jalan, cukup dengan menekan tombol *stop* dan menekan tombol *play* secara berurutan ataupun jika pengguna ingin berhenti di tengah animasi, cukup dengan menekan tombol *pause*.

Komponen utama untuk Animasi terdapat pada `Animation.java` yang me-*generate* Animasi tiap *move* dari suatu State.

Setelah bertemu dengan solusi, GUI pun memanggil `Animation` untuk parameter `List[int] moves`. Kemudian, secara bertahap melakukan *draw* pada *Initial Board* dan memanggil `createMovesAnimation` yang merupakan *wrapper* untuk melakukan iterasi dari seluruh *moves* ke `createMoveAnimation` yang akan menghasilkan Animasi individu dari suatu *moves*.

Tiap `drawBoard` dipanggil, di dalamnya memanggil `drawExit` untuk menampilkan sisi *cell* pinggir mana yang merupakan pintu keluar dan `drawCar` yang menghasilkan grafis berwarna untuk tiap mobil yang ada di *Board*, khusus untuk mobil yang sedang bergerak/melakukan *move* akan terdapat efek tambahan berupa *highlighting* dan *glow* untuk memudahkan pengguna melihat mobil mana yang sedang bergerak.

Setelah *primary car* mencapai pintu keluar, akan dipanggil `createCelebrationAnimation` yang menghasilkan Animasi *primary car* berputar-putar seperti merayakan tibanya *primary car* tersebut ke pintu keluar.

Berikut *sourcecode* dari `Animation.java` dan tidak *include* `GUI.java` karena pada `GUI.java` lebih berisi pada *styling*, sedangkan `Animation.java` memiliki *logic* dan tahapan untuk menghasilkan animasi.

Bab IV
Eksperimen

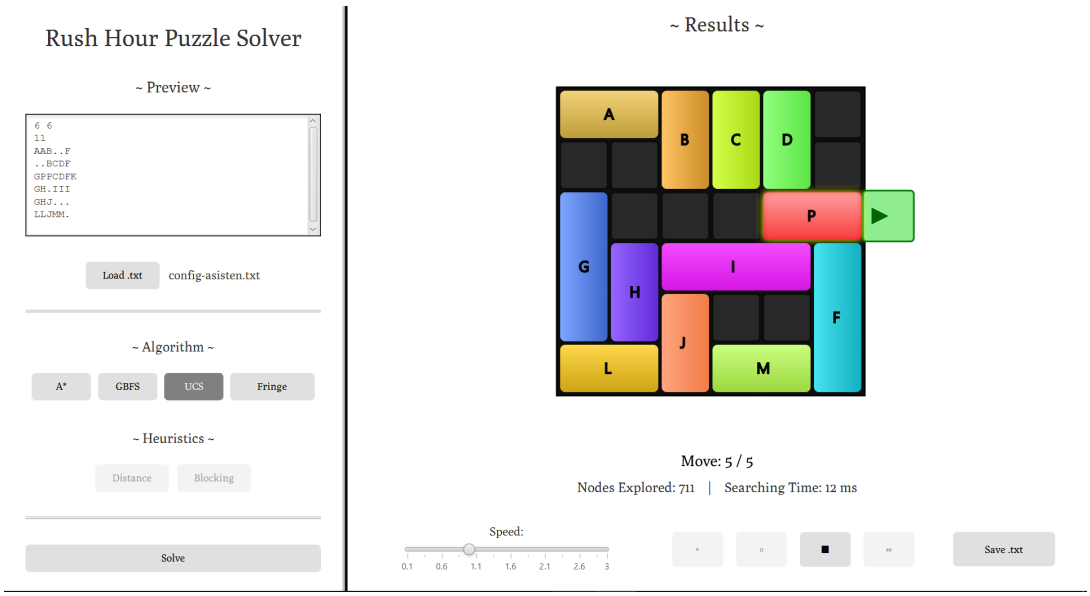
4.1. Eksperimen

Berikut hasil eksperimen dari berbagai tes yang diujikan, mulai *test-case* umum hingga *stress-case* untuk menilai performa program.

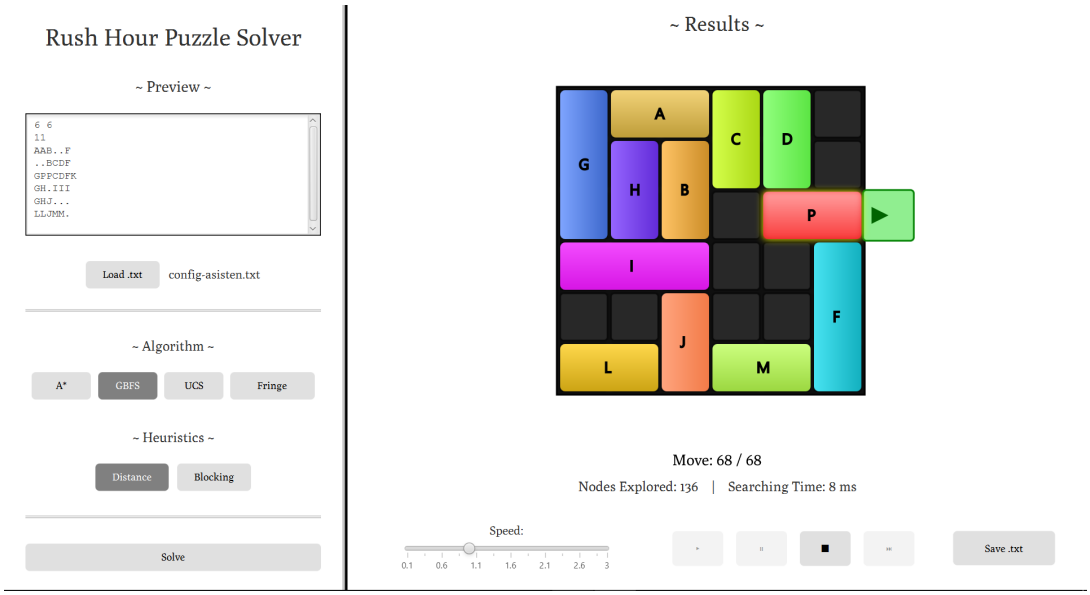
a. Test Case Default dari Asisten

`config-asisten.txt`

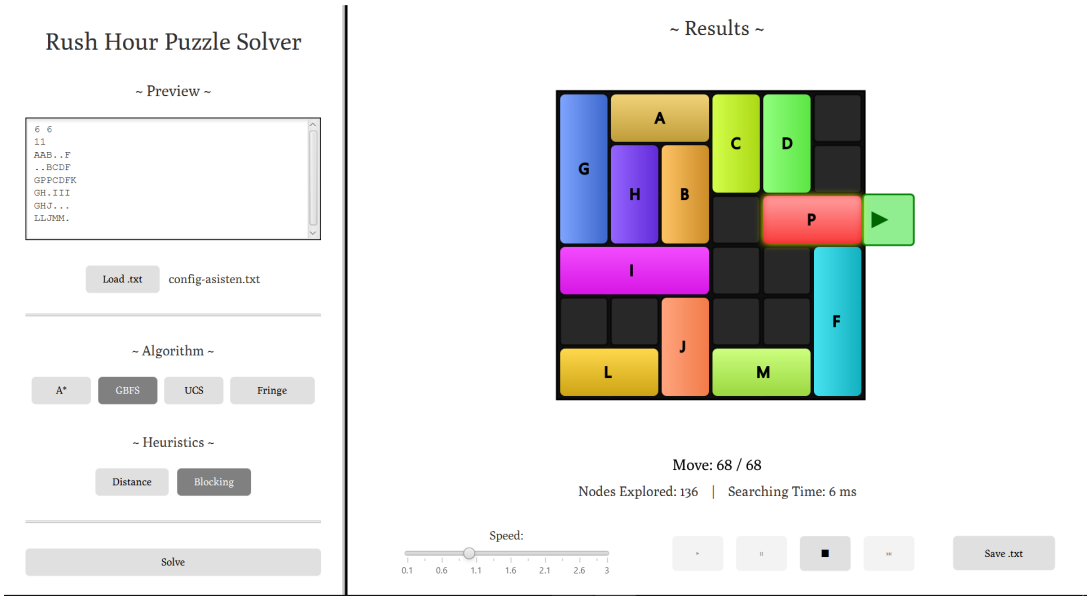
```
6 6
11
AAB..F
..BCDF
GPPCDFK
GH.III
GHJ...
LLJMM.
```



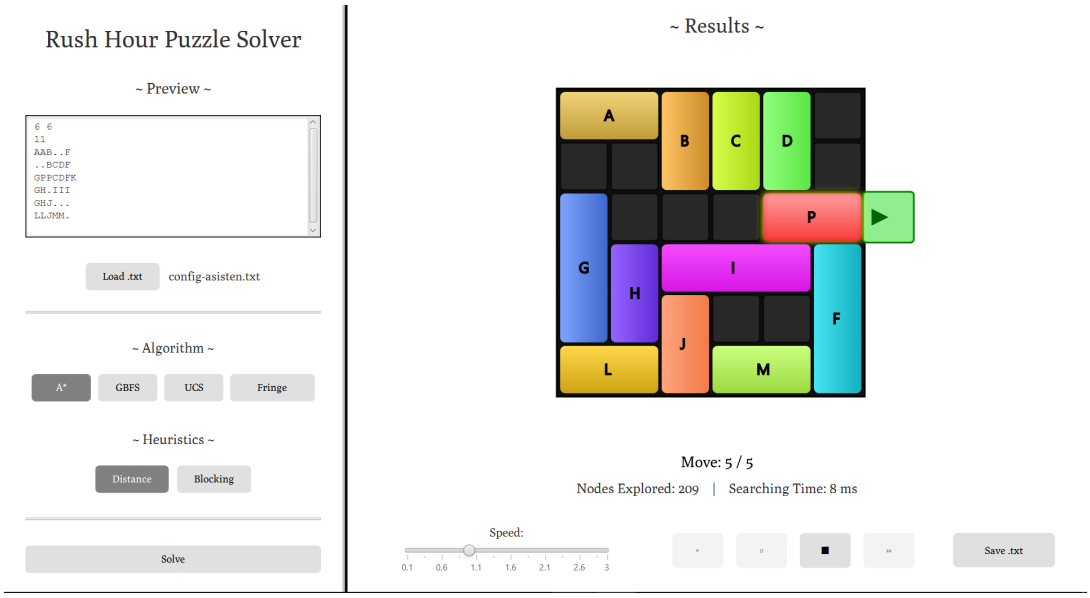
Gambar 9: Percobaan UCS pada TC `config-asisten.txt`



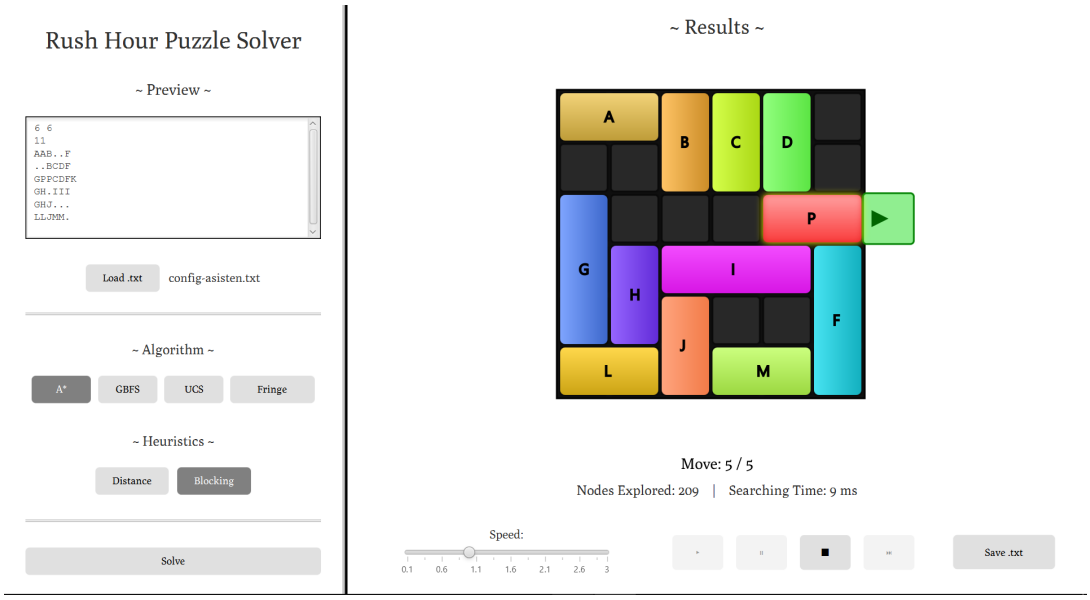
Gambar 10: Percobaan GBFS (Distance) pada TC `config-asisten.txt`



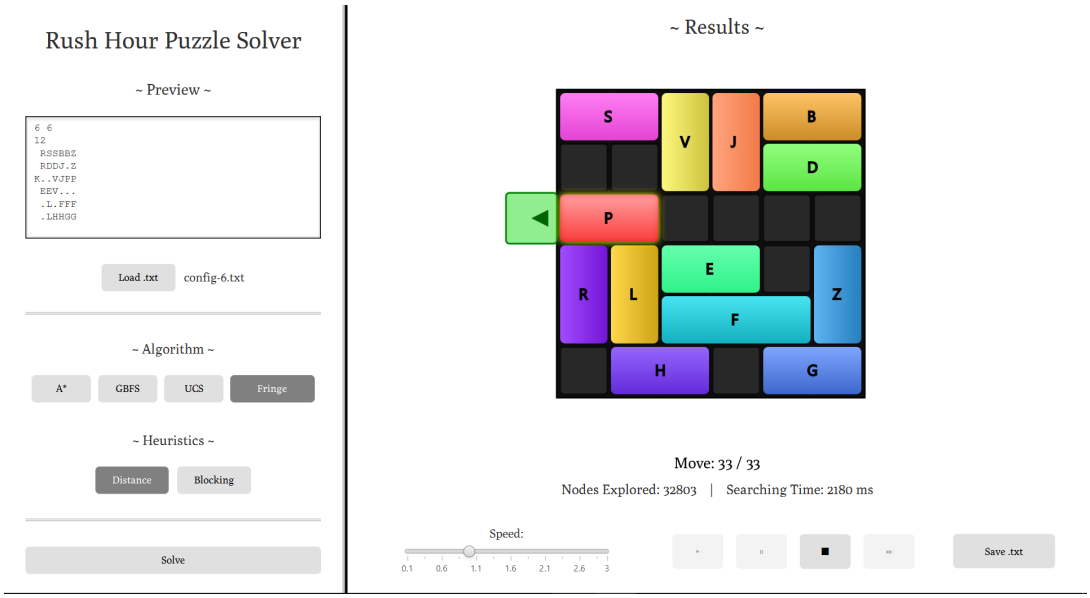
Gambar 11: Percobaan GBFS (Blocking) pada TC `config-asisten.txt`



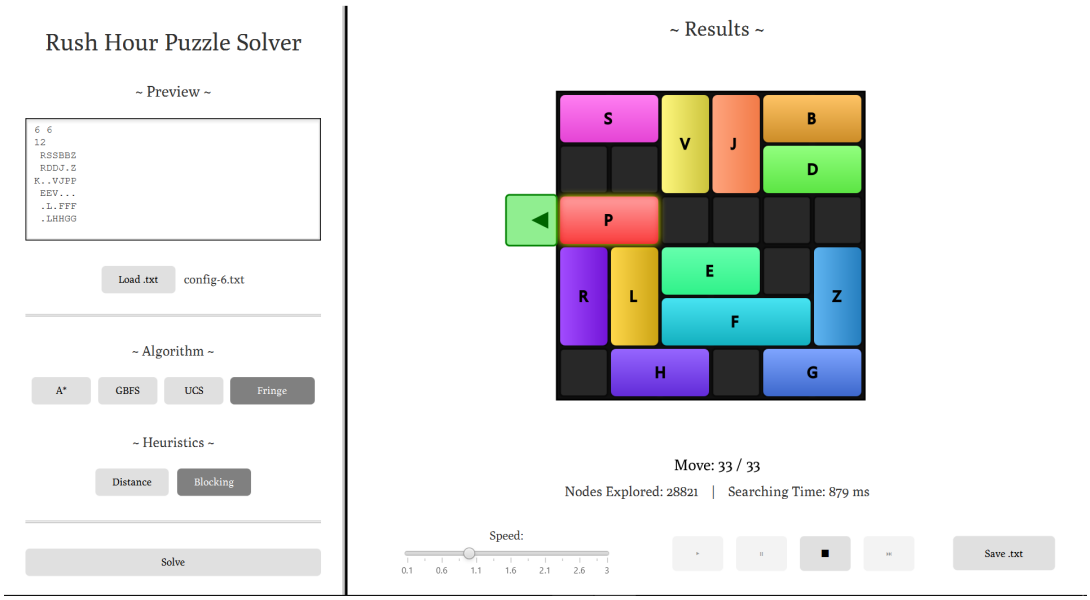
Gambar 12: Percobaan A* (Distance) pada TC `config-asisten.txt`



Gambar 13: Percobaan A* (Blocking) pada TC `config-asisten.txt`



Gambar 14: Percobaan Fringe (Distance) pada TC `config-asisten.txt`

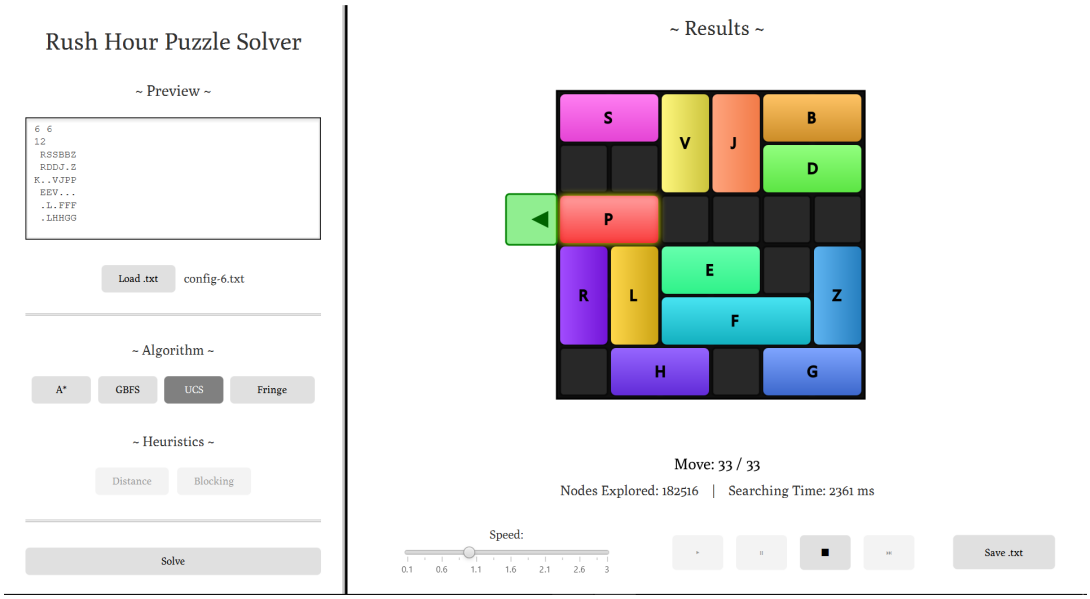


Gambar 15: Percobaan Fringe (Blocking) pada TC `config-asisten.txt`

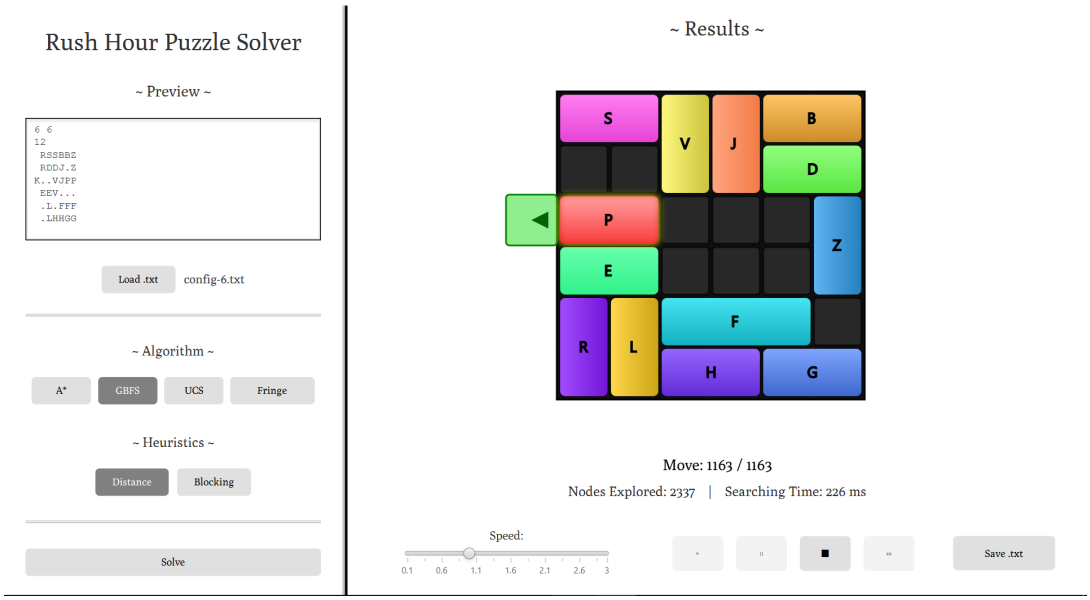
b. Test Case 6 x 6

config-6.txt

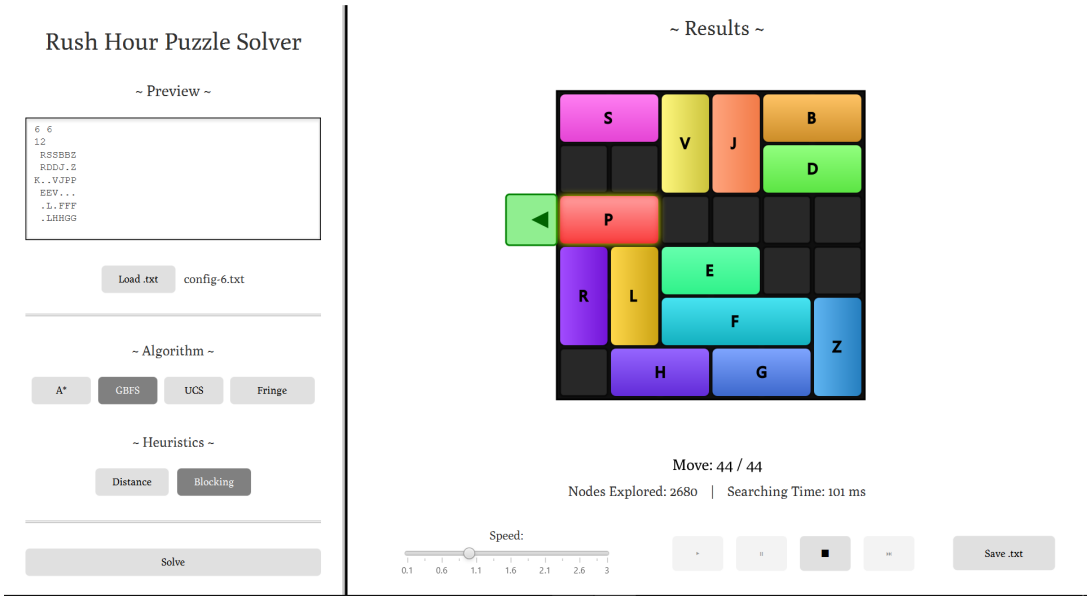
```
6 6
12
 RSSBBZ
 RDDJ.Z
K..VJPP
 EEV...
 .L.FFF
 .LHHGG
```



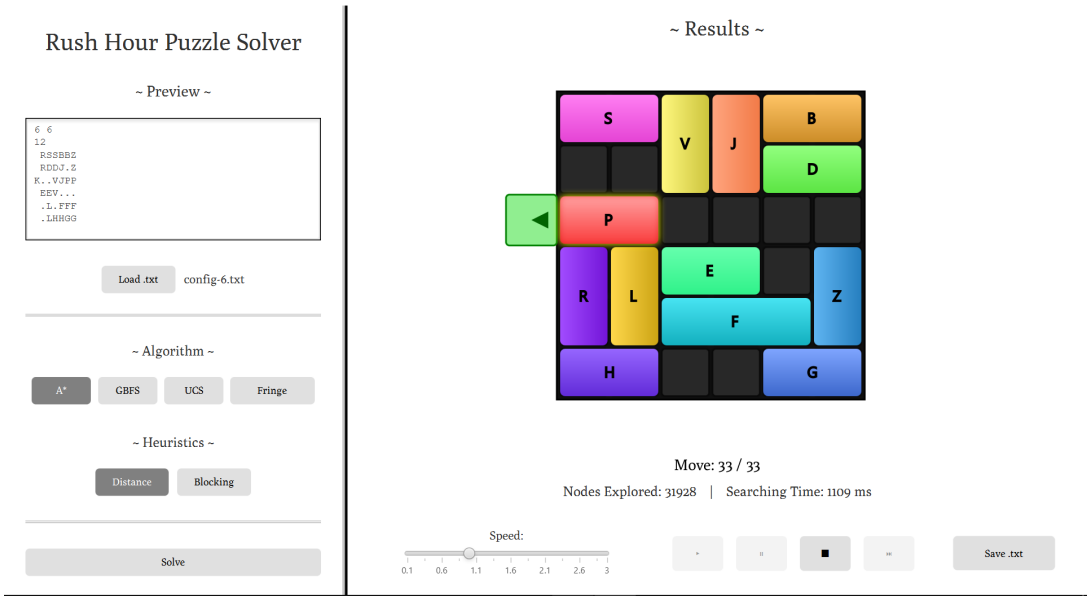
Gambar 16: Percobaan UCS pada TC `config-6.txt`



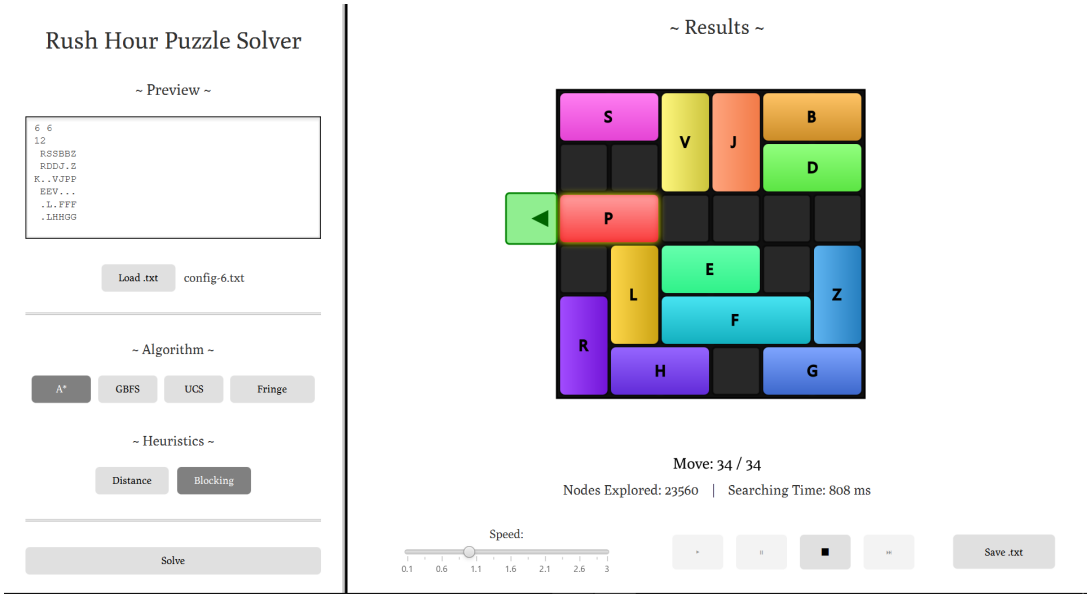
Gambar 17: Percobaan GBFS (Distance) pada TC `config-6.txt`



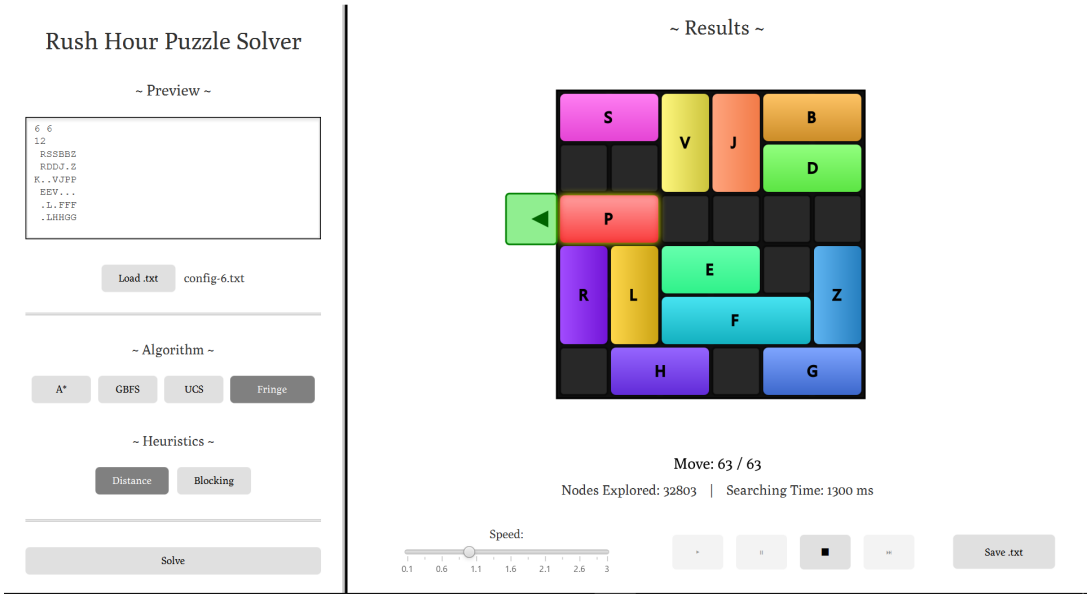
Gambar 18: Percobaan GBFS (Blocking) pada TC `config-6.txt`



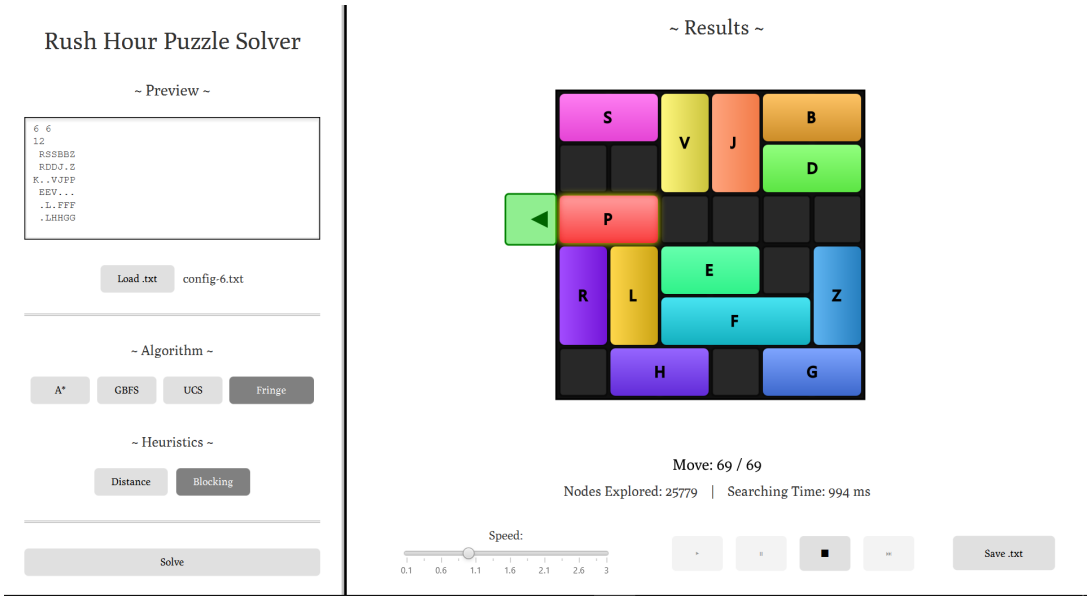
Gambar 19: Percobaan A* (Distance) pada TC `config-6.txt`



Gambar 20: Percobaan A* (Blocking) pada TC `config-6.txt`



Gambar 21: Percobaan Fringe (Distance) pada TC `config-6.txt`

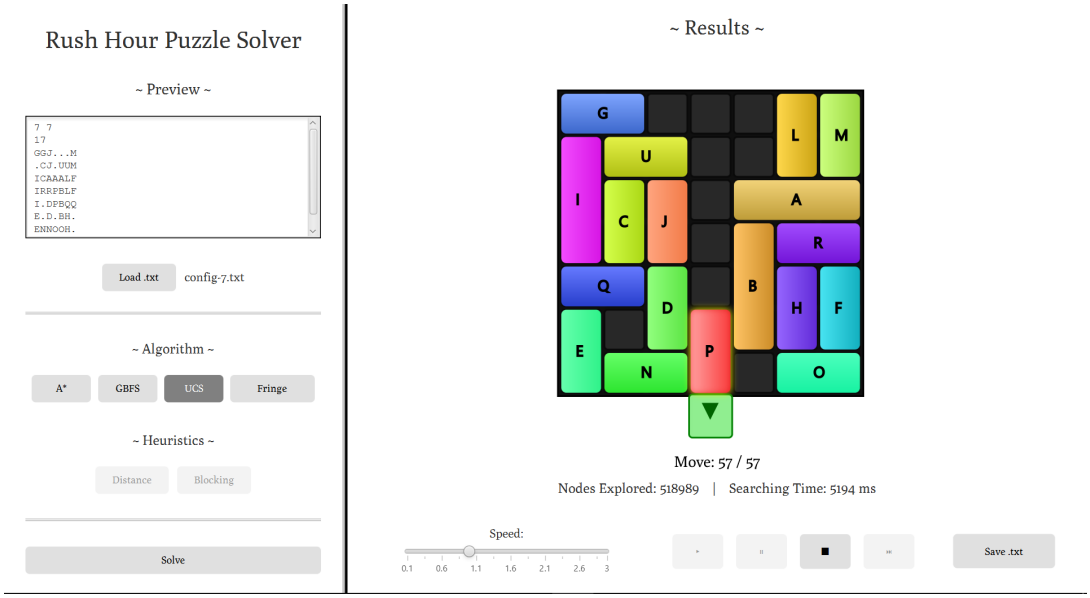


Gambar 22: Percobaan Fringe (Blocking) pada TC `config-6.txt`

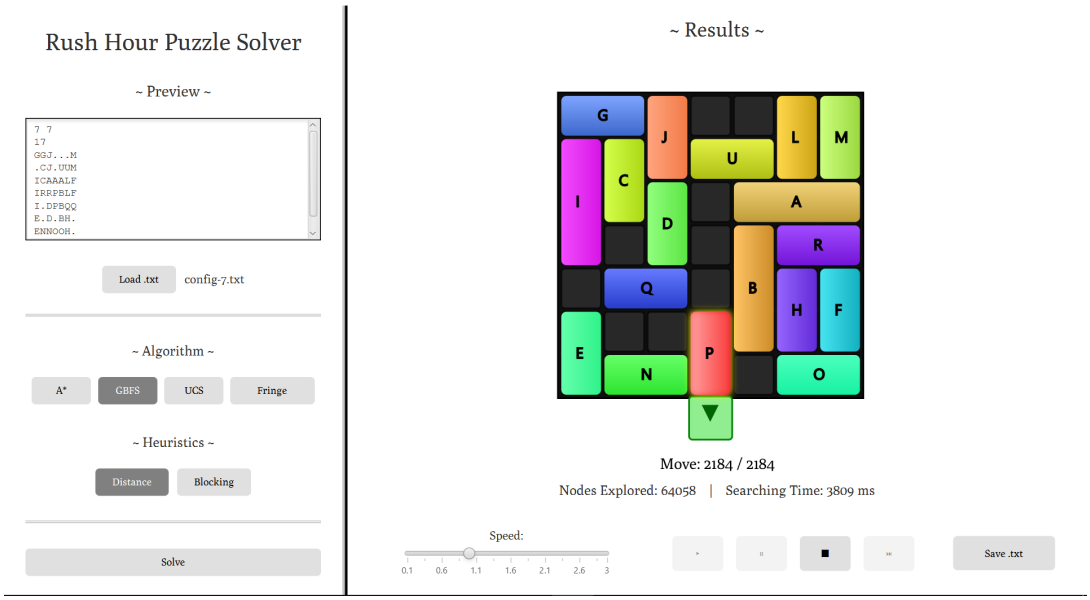
b. Test Case 7 x 7

config-6.txt

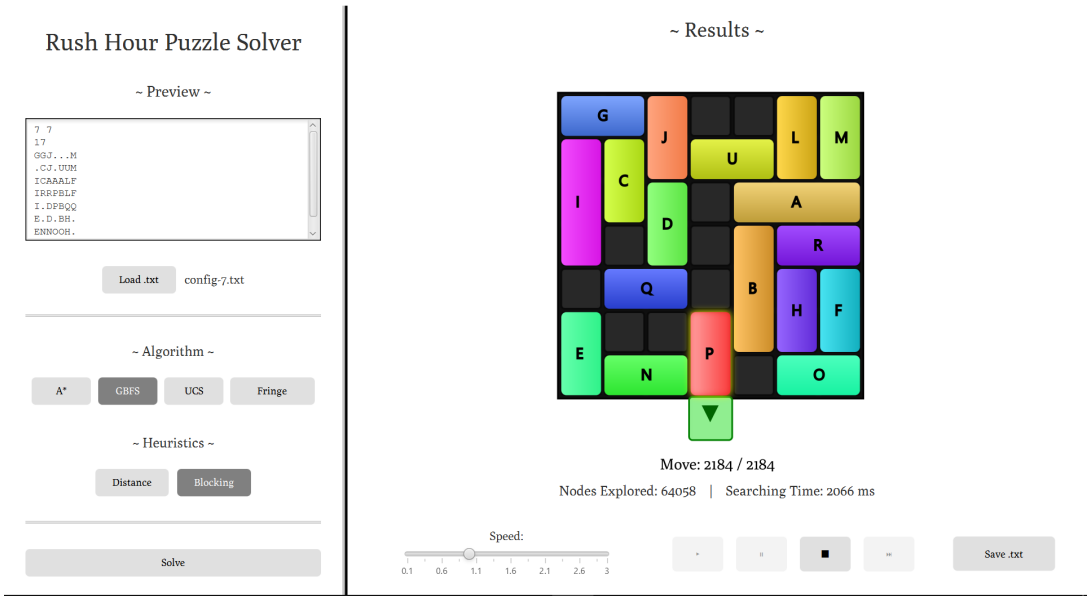
```
7 7
17
GGJ...M
.CJ.UUM
ICAAALF
IRRPBLF
I.DPBQQ
E.D.BH.
ENNOOH.
   K
```



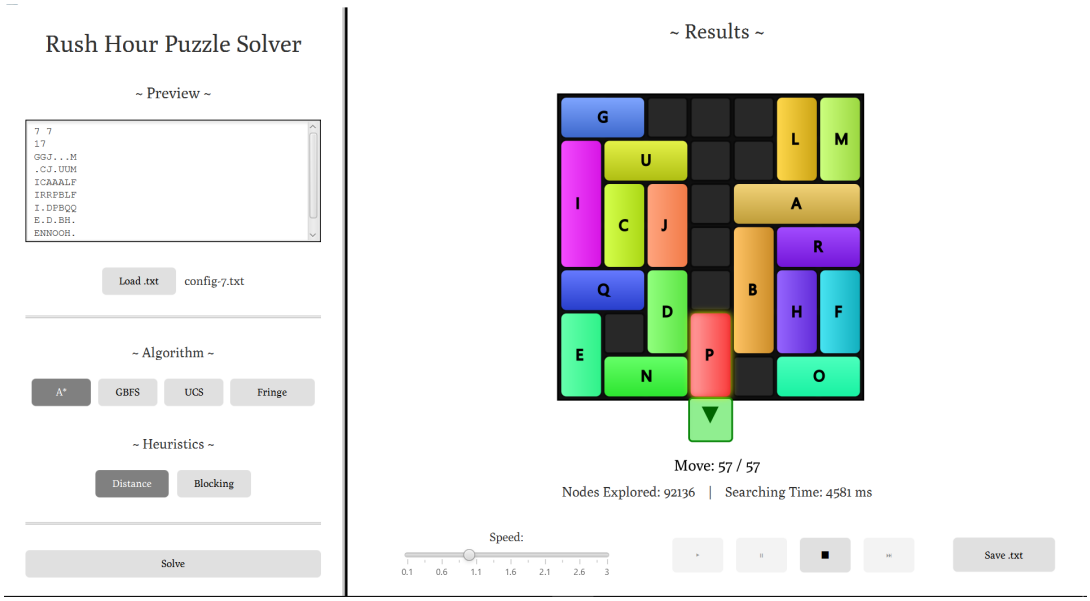
Gambar 23: Percobaan UCS pada TC `config-7.txt`



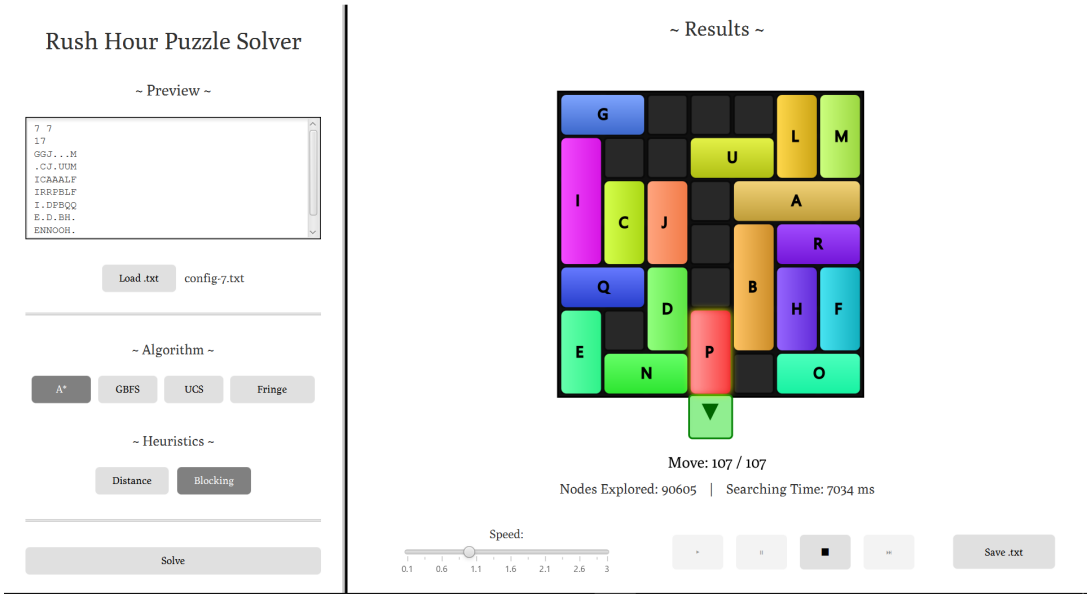
Gambar 24: Percobaan GBFS (Distance) pada TC `config-7.txt`



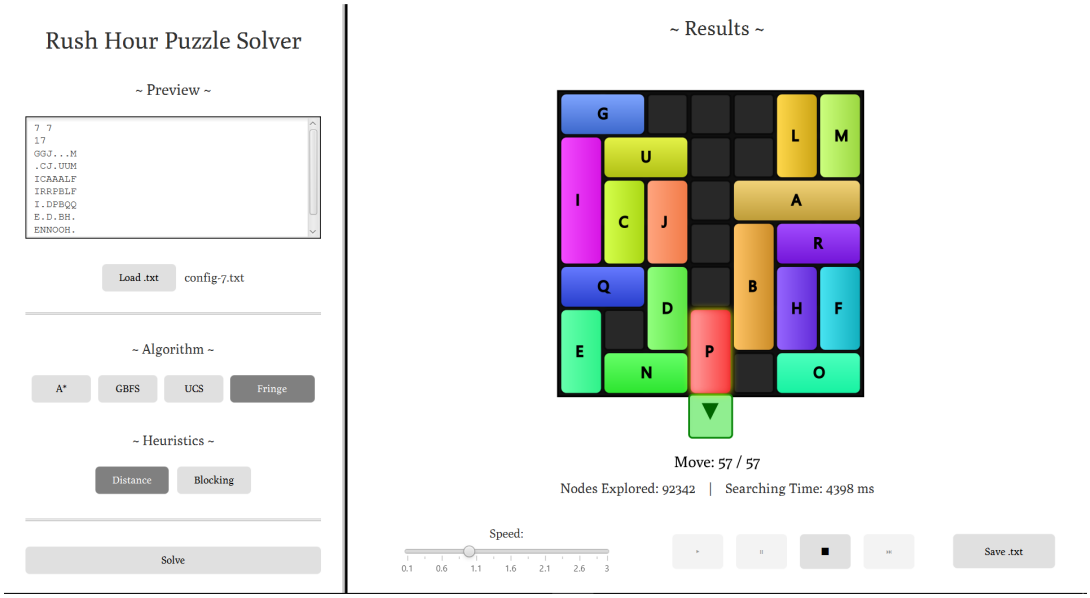
Gambar 25: Percobaan GBFS (Blocking) pada TC `config-7.txt`



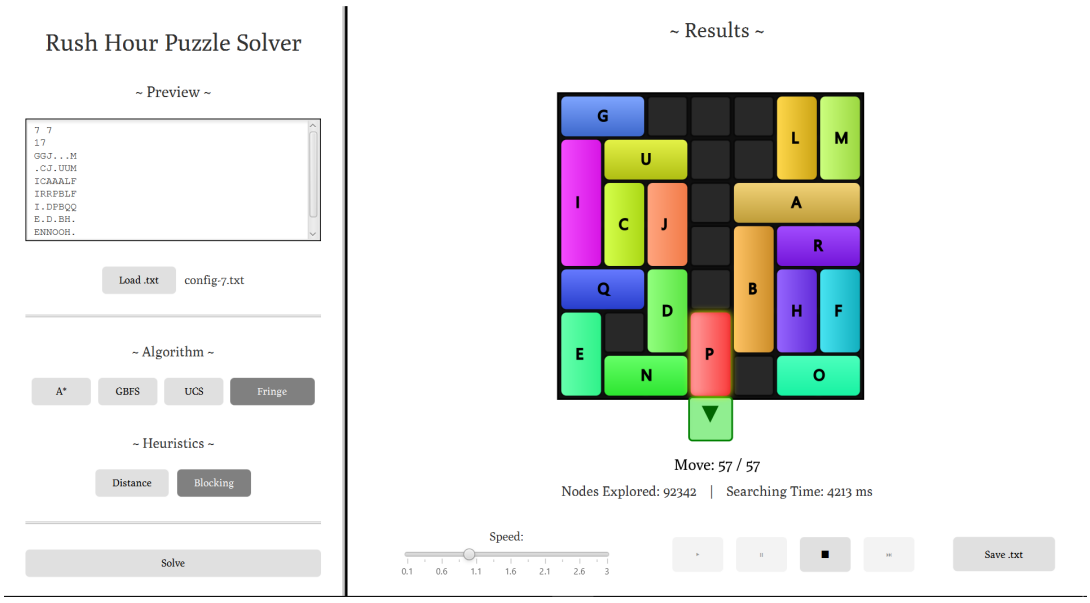
Gambar 26: Percobaan A* (Distance) pada TC `config-7.txt`



Gambar 27: Percobaan A* (Blocking) pada TC `config-7.txt`



Gambar 28: Percobaan Fringe (Distance) pada TC `config-7.txt`

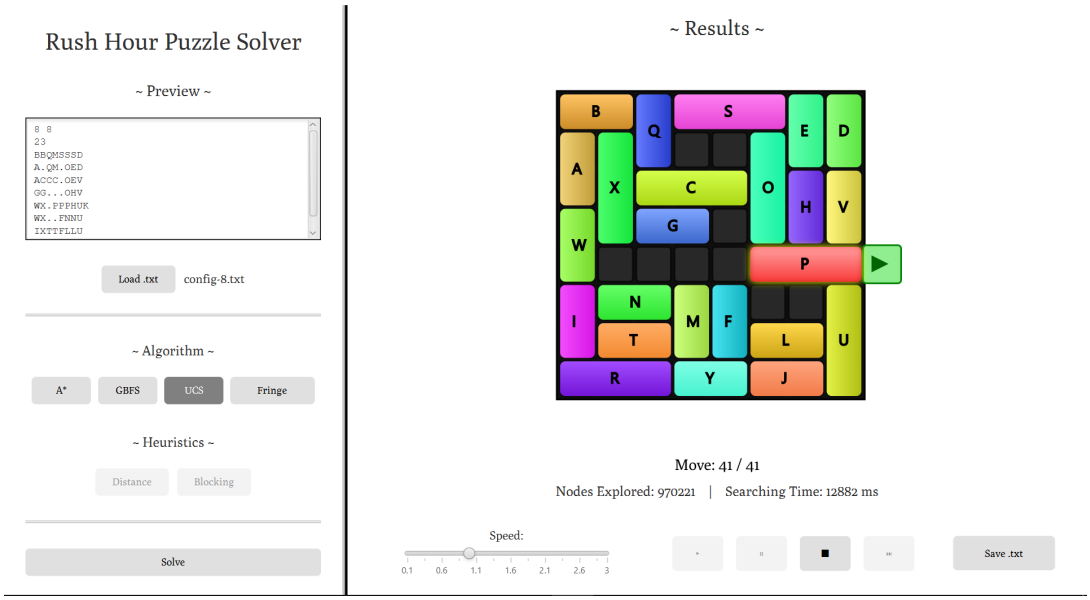


Gambar 29: Percobaan Fringe (Blocking) pada TC `config-7.txt`

b. **Test Case 8 x 8 (*Stress Case*)**

`config-8.txt`

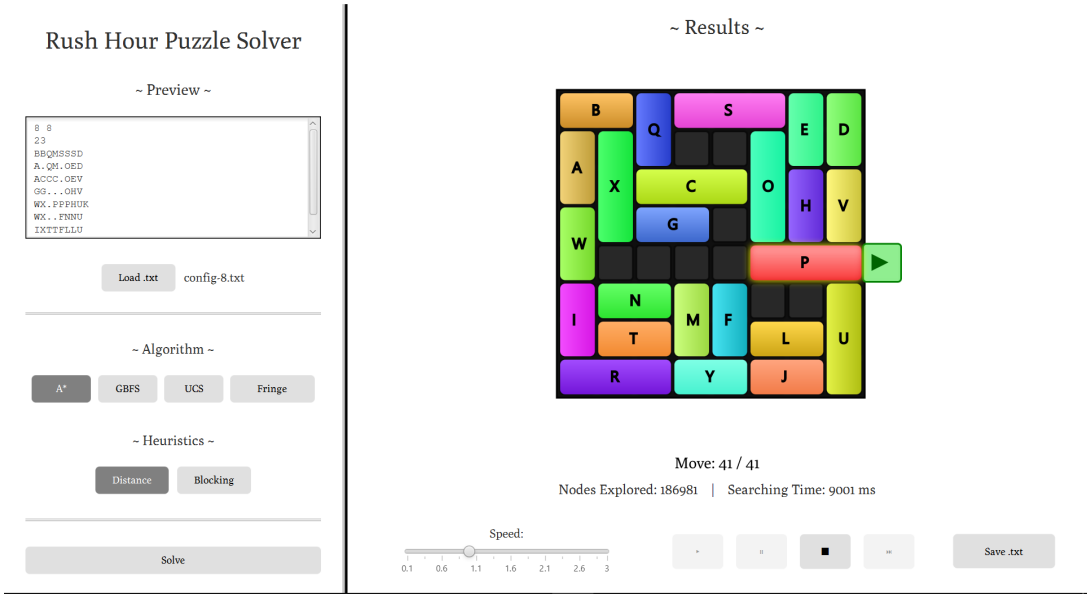
```
8 8
23
BBQMSSSD
A.QM.OED
ACCC.OEV
GG...OHV
WX.PPPHUK
WX..FNNU
IXTTFLLU
IRRRYYJJ
```



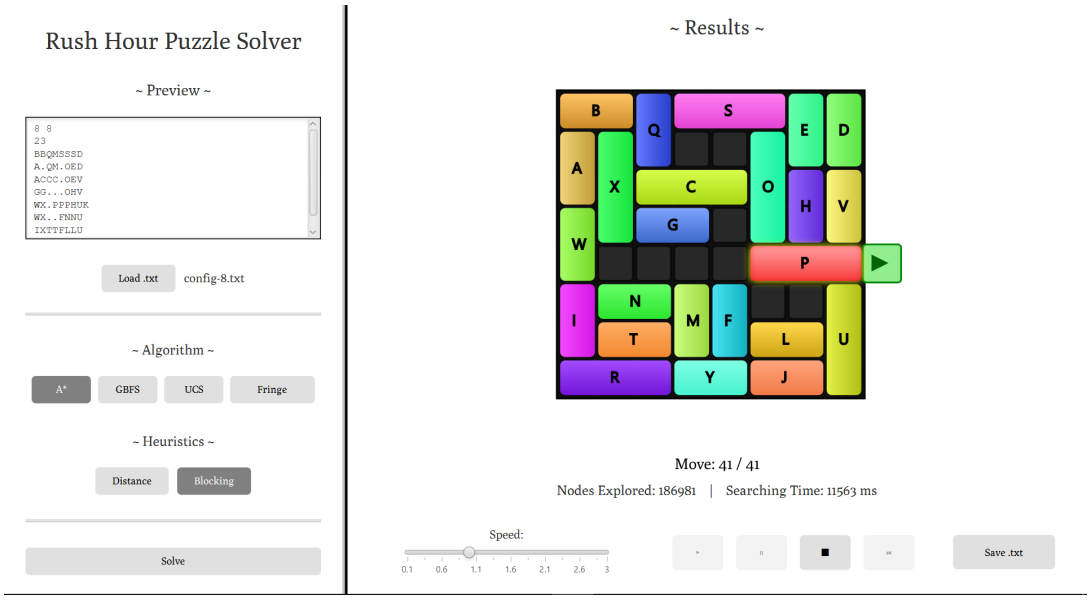
Gambar 30: Percobaan UCS pada TC `config-8.txt`

Gambar 31: Percobaan GBFS (Distance) pada TC `config-8.txt`

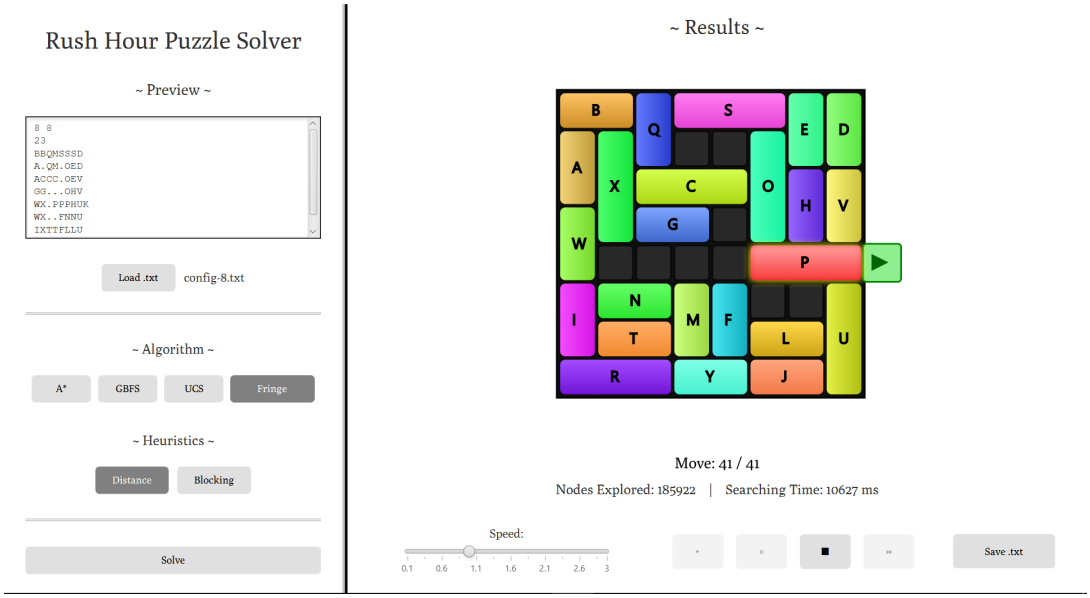
Gambar 32: Percobaan GBFS (Blocking) pada TC `config-8.txt`



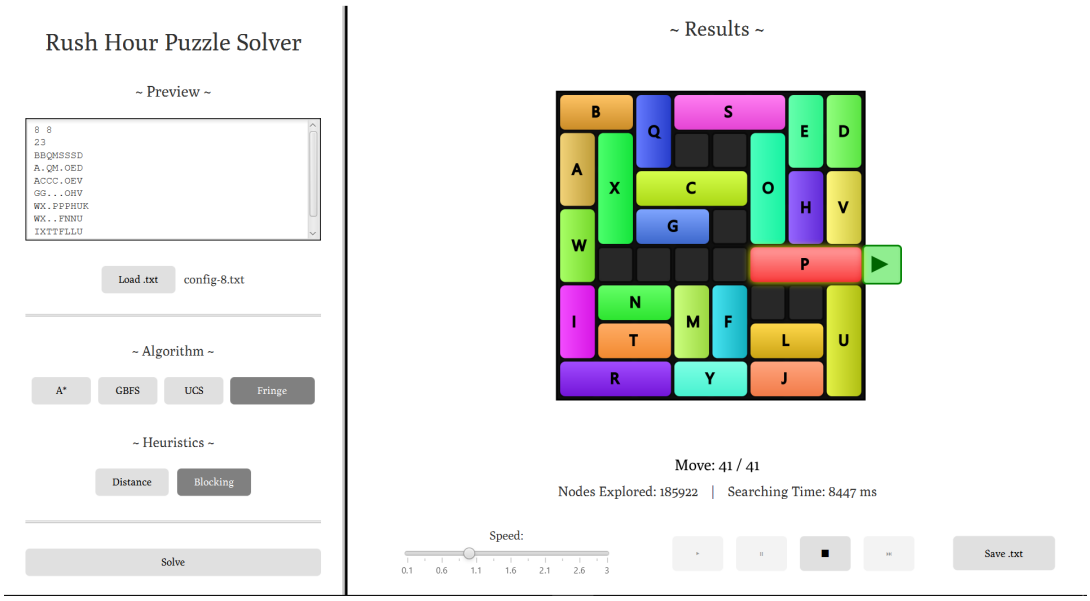
Gambar 33: Percobaan A* (Distance) pada TC `config-8.txt`



Gambar 34: Percobaan A* (Blocking) pada TC `config-8.txt`



Gambar 35: Percobaan Fringe (Distance) pada TC `config-8.txt`

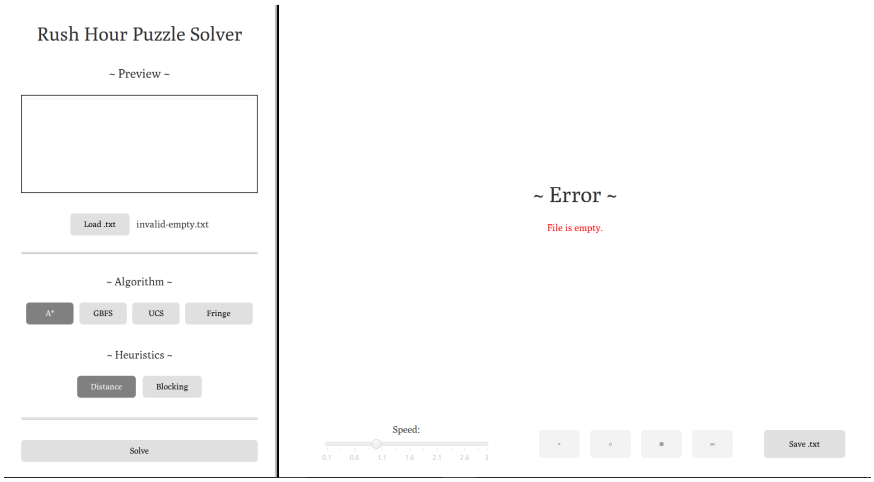


Gambar 36: Percobaan Fringe (Blocking) pada TC `config-8.txt`

b. Kasus-kasus Invalid

`invalid-empty.txt`

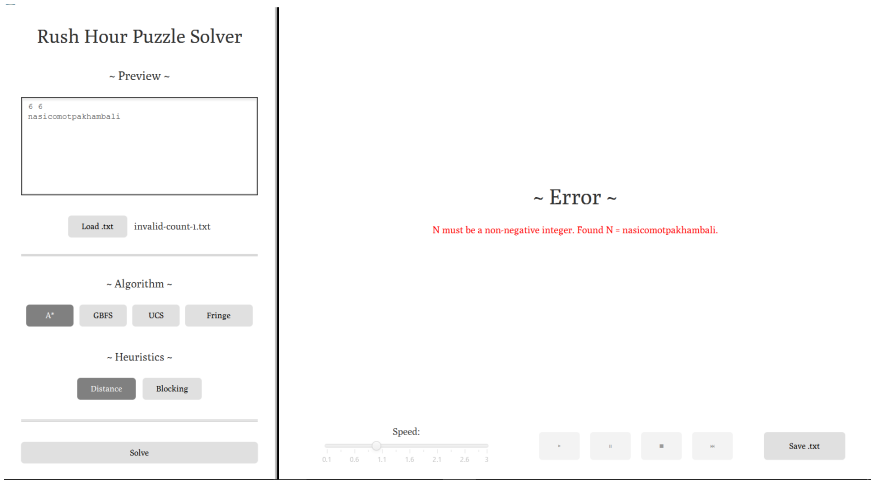
```
```



Gambar 37: Percobaan Empty File pada TC `invalid-empty.txt`

`invalid-count-1.txt`

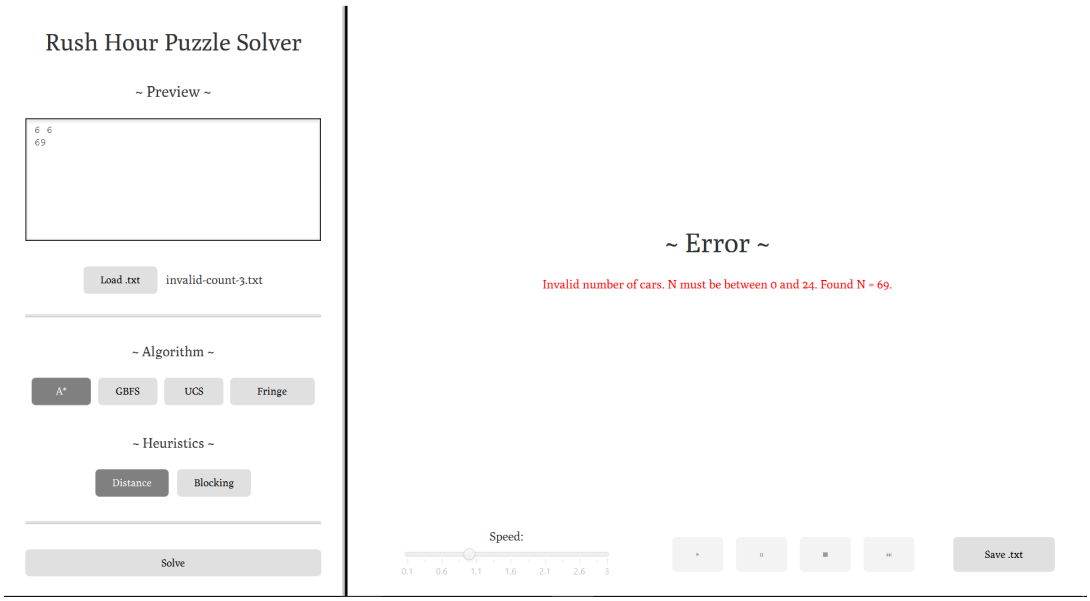
```
6 6
nasicomotpakhambali
```



Gambar 38: Percobaan Invalid Num Primary `invalid-count-1.txt`

invalid-count-2.txt

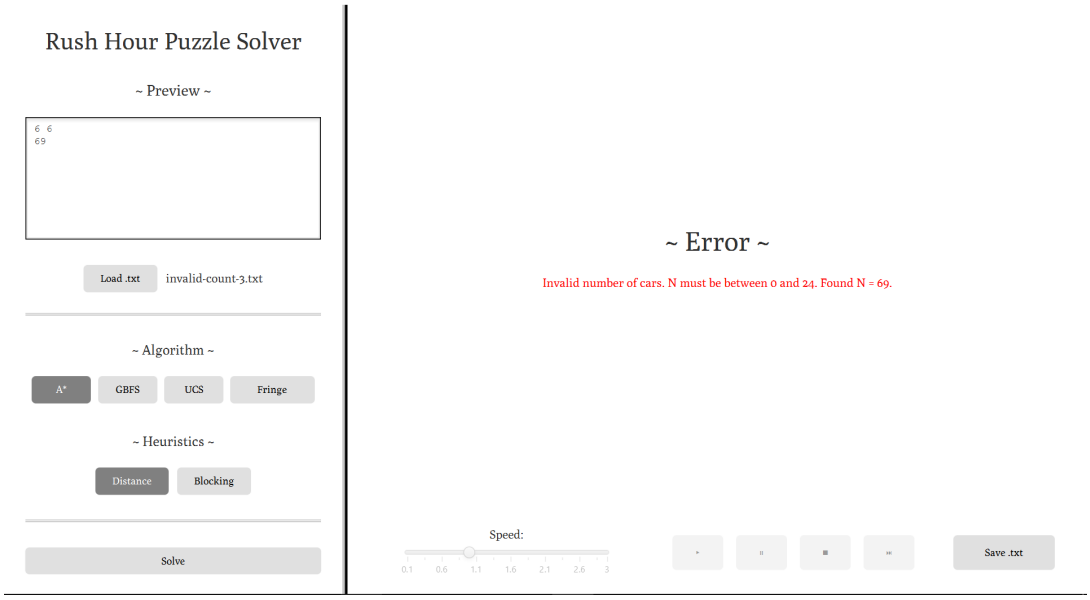
```
6 6
```



Gambar 39: Percobaan Invalid Num Primary `invalid-count-2.txt`

invalid-count-3.txt

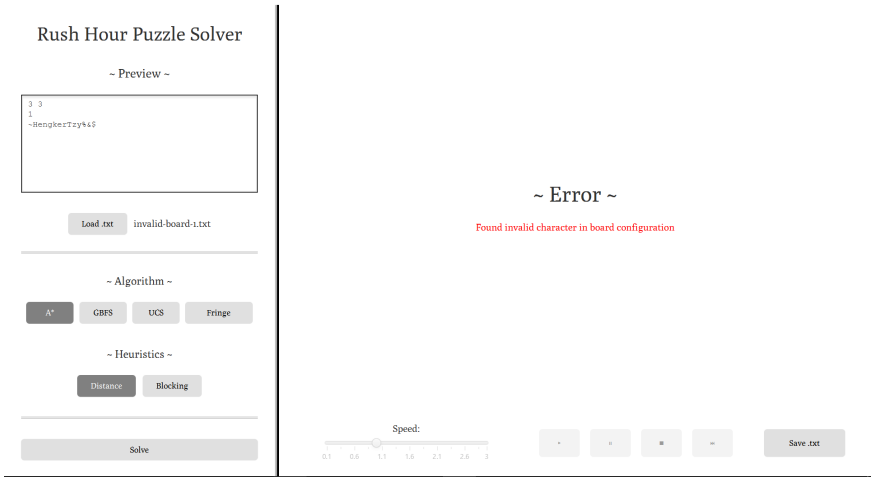
```
6 6
69
```



Gambar 40: Percobaan Invalid Num Primary `invalid-count-3.txt`

invalid-board-1.txt

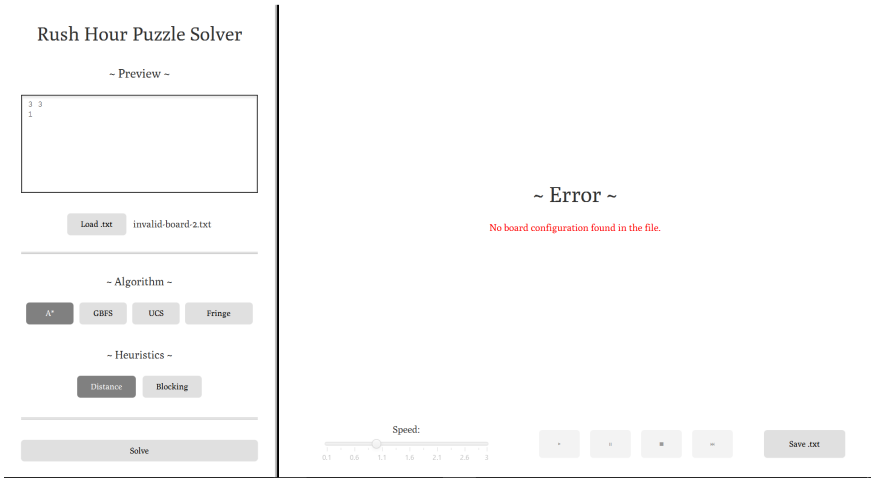
```
3 3
1
~HengkerTzy%&$
```



Gambar 41: Percobaan Invalid Board `invalid-board-1.txt`

invalid-board-2.txt

```
3 3
1
```



Gambar 42: Percobaan Invalid Board `invalid-board-2.txt`

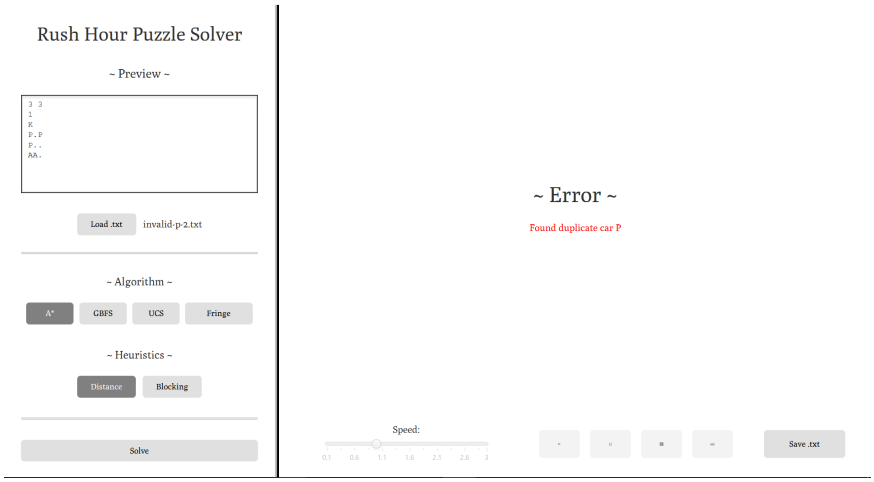
invalid-p-1.txt

```
3 3
1
...
...
AA.
```

Gambar 43: Percobaan Invalid P `invalid-p-1.txt`

invalid-p-2.txt

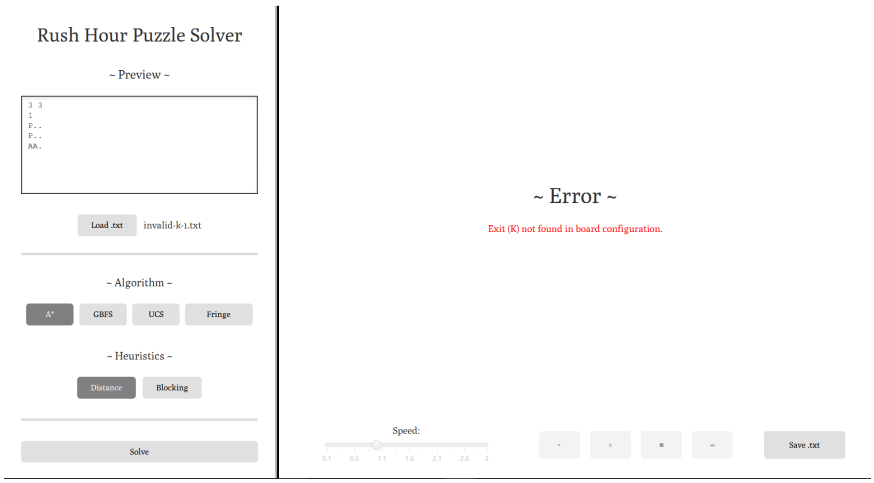
```
3 3
1
K
P.P
P..
AA.
```



Gambar 44: Percobaan Invalid P `invalid-p-2.txt`

invalid-k-1.txt

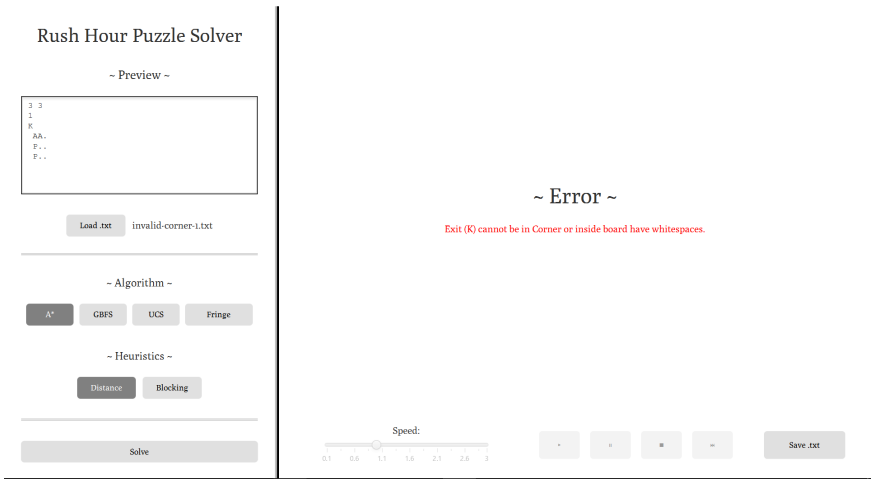
```
3 3
1
...
...
AA.
```



Gambar 45: Percobaan Invalid K `invalid-k-1.txt`

invalid-corner-1.txt

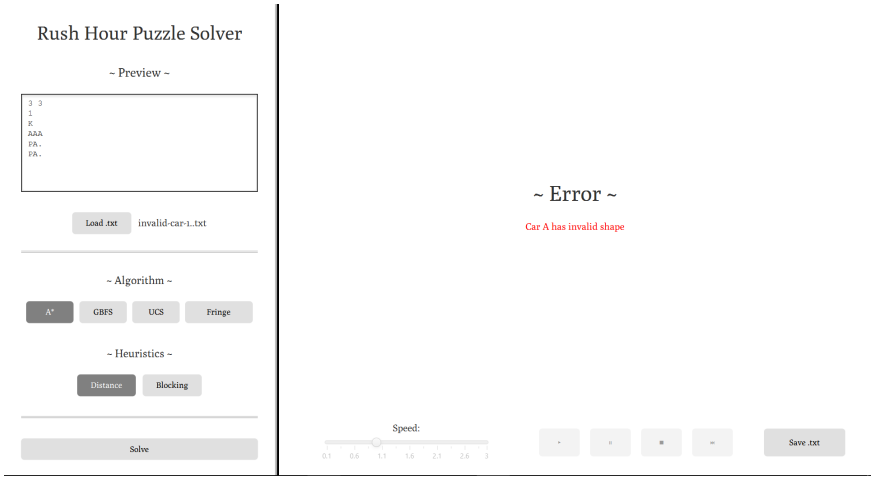
```
3 3
1
K
P.P
P..
AA.
```



Gambar 46: Percobaan Invalid K `invalid-corner-1.txt`

invalid-car-1.txt

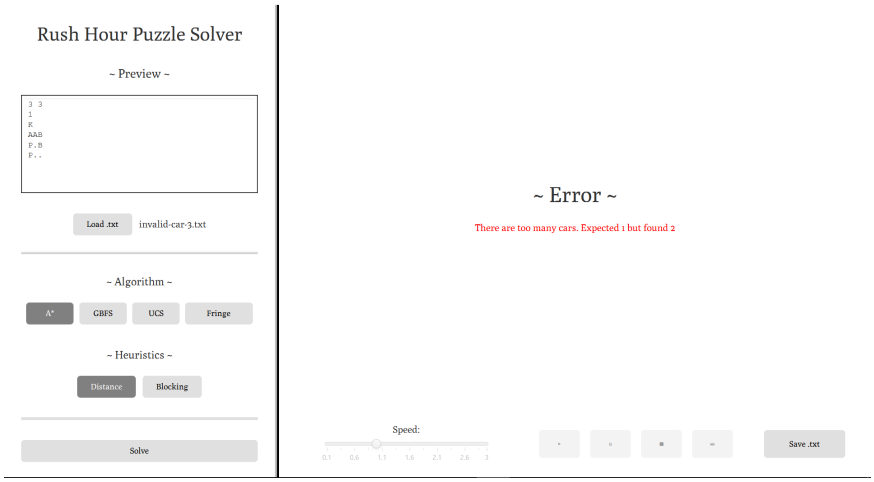
```
3 3
1
K
AAA
PA.
PA.
```



Gambar 47: Percobaan Mobil Invalid `invalid-car-1.txt`

invalid-car-3.txt

```
3 3
1
K
AAB
P.B
P..
```



Gambar 48: Percobaan Mobil Invalid `invalid-car-3.txt`

1.2. Analisis dan Pembahasan

Berdasarkan percobaan yang telah dilakukan, didapat bahwa `UCS`, `A* Search`, `Fringe Search` dengan *heuristic* yang baik dapat selalu mendapatkan solusi yang optimal, sedangkan `GBFS` tidak selalu berhasil mendapatkan solusi optimal, terkadang mendapatkan solusi sub-optimal.

Dari pengamatan juga, didapatkan bahwa `UCS` sering kali membangkit simpul yang jauh lebih banyak jika dibandingkan dengan algoritma lainnya. Selain itu, didapatkan bahwa jumlah simpul yang dibangkitkan antara `A* Search` dan `Fringe Search` tidak beda jauh.

Selain itu, terdapat hal yang unik terjadi pada GBFS saat *test case* 8 x 8. GBFS tidak dapat menemukan solusi di saat yang lain dengan mudah menemukan solusinya. Hal ini mungkin saja terjadi karena GBFS "tertipu/terjebak" oleh pendekatannya yang selalu mengambil *cost* terendah pada saat itu, tanpa melihat belakang.

Analisis lebih lanjut untuk tiap algoritma, dipaparkan sebagai berikut:

1.2.1 Uniform Cost Search

`Uniform Cost Search` memperluas simpul berdasarkan biaya kumulatif $g(n)$, dan pada kasus Rush Hour, di mana setiap langkah memiliki biaya yang seragam (1), algoritma ini menjamin solusi dengan jumlah langkah minimal. UCS menggunakan *priority queue* yang diurutkan berdasarkan $g(n)$, sehingga algoritma ini mengeksplorasi seluruh konfigurasi papan sampai solusi optimal ditemukan. Dari segi kompleksitas, UCS memiliki waktu dan ruang $O(b)$, dengan b adalah faktor percabangan dan d adalah kedalaman solusi. Implementasi kami juga mencatat jumlah simpul yang dikunjungi dan menggunakan *visitedStates* untuk menghindari siklus. Meskipun lengkap dan optimal, UCS menjadi sangat mahal pada *puzzle* dengan ruang status besar karena *frontier* yang tumbuh secara eksponensial.

1.2.2 Greedy Best First Search

`Greedy Best First Search` memperluas simpul hanya berdasarkan nilai *heuristic* $h(n)$ saja tanpa memperhitungkan *cost* sejauh ini dalam graf. Sehingga algoritma ini tampak mengeksplor papan secara *greedy* menuju solusi. Dalam implementasi `Greedy Best First Search` sangat cepat dan hemat memori, *runtime*-nya bisa sangat cepat. Dalam eksekusi, biasanya didapat jauh lebih baik dibandingkan batas terburuknya yaitu, $O(b)$ dengan m adalah kedalaman maksimal. Jika dipasangkan dengan *heuristic* yang baik, dapat mempercepat *runtime* serta ruang pencarian dapat dihemat sebab *heuristic* yang baik dapat memangkas ruang pencarian yang tidak perlu yang berakibat algoritma semakin efisien. Namun, secara teoritis `Greedy Best First Search` tidak dapat menjamin jalur solusi yang optimal, karena dengan hanya mempertimbangkan nilai *heuristic* saja, dapat membuat ia terjebak dalam solusi *sub-optimal* dimana tampak menjanjikan, tetapi memerlukan banyak langkah secara keseluruhan.

1.2.3 A* Search

`A* Search` memperluas simpul berdasarkan nilai *f-value* dimana $f(n) = h(n) + g(n)$ dimana $h(n)$ merupakan nilai *heuristic* dan $g(n)$ merupakan *cost* sejauh ini. Dengan memasangkannya dengan *heuristic* yang *admissible* dapat menjamin solusi optimal dengan ruang pencarian yang efisien dibandingkan dengan `Uniform Cost Search`. Secara teoritis kompleksitas `A* Search` adalah $O(b)$, namun secara empiris, `A* Search` mengunjungi jauh lebih sedikit simpul karena *heuristic* membantu memandu pencarian. Ruang yang digunakan sebanding dengan jumlah simpul yang dikunjungi, dan waktu eksekusi keseluruhan biasanya setengah atau lebih cepat dibandingkan UCS pada *puzzle* dengan ukuran menengah.

1.2.4 Fringe Search

`Fringe Search` mirip dengan `A* Search`, tetapi meminimalkan penggunaan memori dan *overhead heap* pada *priority queue*. Alih-alih menyimpan semua *frontier* dalam *priority queue*, `Fringe Search` menggunakan *list* dan membatasi ruang pencarian pada suatu fase dengan *value f-limit* dan ditingkat per fase. Pendekatan ini memungkinkan untuk mengurangi o*overhead* memori dan kompleksitas yang dibawa oleh *priority queue*. Secara teoritis, `Fringe Search` memiliki kompleksitas terburuk sama dengan `A* Search` yaitu $O(b)$. Dengan *heuristic* yang *admissible* atau cukup informatif, `Fringe Search` dapat memiliki performa yang sama dengan `A* Search` tapi memiliki ruang memori yang lebih sedikit jika dibandingkan dengan `A* Search`.

Seluruh analisis di atas dapat dirangkum menjadi tabel berikut:

Tabel 1: Perbandingan Algoritma Pathfinding

Nama	Time-Complexity	Space-Complexity	Optimal	Lengkap
UCS	$O(b^d)$	$O(b^d)$	✓	✓
GBFS	$O(b^m)$	$O(b^m)$		
A*	$O(b^d)$	$O(b^d)$	✓	✓
Fringe Search	$O(b^d)$	$O(bd)$	✓	✓

Bab V
Penutup

5.1. Kesimpulan

Dalam proyek Tugas Kecil 3 IF2211 Strategi Algoritma ini, algoritma *pathfinding* berupa UCS, GBFS, A*, dan Fringe Search dapat digunakan dalam penyelesaian Rush Hour Puzzle dengan memberikan hasil yang baik secara cepat dan efisien dengan *trade off*-nya masing-masing.

Kami berhasil untuk mengimplementasikan algoritma *pathfinding* dengan struktur data *Priority Queue* dalam program penyelesaian Rush Hour kami. Selain menyediakan algoritma *pathfinding* yang wajib diimplementasikan, program kami juga mengimplementasikan algoritma *pathfinding* dan heuristic tambahan.

Selain itu, pengguna dapat dengan mudah melihat jalur pergerakan/*moves* yang terbentuk dalam *path* melalui *Graphical User Interface* yang diimplementasikan pada program kami. Cukup dengan melakukan *load* konfigurasi, program pun akan menghasilkan animasi jalur pergerakan menuju solusi.

5.2. Saran

Pelaksanaan Tugas Kecil 3 IF2211 Strategi Algoritma di Semester II (Genap) Tahun 2024/2025 merupakan pengalaman yang sangat berharga bagi kami. Dari pengalaman ini, kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

rzi

Tantangan terbesar sebenarnya dalam tucil dan tubes di IF hanyalah prokas. Sungguh, dunia begitu damai jika kelar tucil dan tubes jauh hari sebelum deadline. Terima kasih banyak adha suami hutao, semoga yang disemogakan secepatnya tersemogakan.

adharid

Berikanlah aku eyeglasses wolfcut chindo huzz gf ya allah.

Semoga saran-saran ini membantu pembaca dalam menyiapkan diri untuk menangani tugas serupa di masa depan.

5.3. Tautan

Repository program dan GIF yang dihasilkan dapat diakses melalui tautan berikut:

`https://github.com/zirachw/Tucil3_13523004_13523098`

5.4. Lampiran

Tabel 2: Tabel Spesifikasi Tugas Kecil 2

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt dan menyimpan solusi berupa print board tahap per tahap dalam berkas .txt	✓	
5	[**Bonus**] Implementasi algoritma pathfinding alternatif	✓	
6	[**Bonus**] Implementasi 2 atau lebih heuristik alternatif	✓	
7	[**Bonus**] Program memiliki GUI	✓	
8	Program dan laporan dibuat (kelompok) sendiri	✓	

Referensi

Rinaldi Munir. (2025). "Penentuan rute (Route/Path Planning) - Bagian 1" Diakses pada 19 Mei 2025. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/21-Route-Planning-(2025)-Bagian1.pdf

Rinaldi Munir. (2025). "Penentuan rute (Route/Path Planning) - Bagian 2" Diakses pada 19 Mei 2025. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/22-Route-Planning-(2025)-Bagian2.pdf

Educative. (2025). "What is uniform-cost search?" Diakses pada 20 Mei 2025. https://www.educative.io/answers/what-is-uniform-cost-search

Educative. (2025). "What is uniform-cost search?" Diakses pada 20 Mei 2025. https://www.educative.io/answers/what-is-uniform-cost-search

K. Mahathi. (2024). "Informed Search-Greedy Best First Search" Diakses pada 20 Mei 2025. https://medium.com/@2200032841cseh/informed-search-greedy-best-first-search-b0ea599aa718

codeacademy. (2023). "Greedy Best-First Search" Diakses pada 20 Mei 2025. https://www.educative.io/answers/what-is-uniform-cost-search

Geeks For Geeks. (2024). "A* algorithm and its Heuristic Search Strategy in Artificial Intelligence" Diakses pada 20 Mei 2025. https://www.geeksforgeeks.org/a-algorithm-and-its-heuristic-search-strategy-in-artificial-intelligence/

Markus Enzenberger. (2005). "Fringe Search: Beating A* at Pathfinding on Game Maps" Diakses pada 28 Maret 2025. https://webdocs.cs.ualberta.ca/~holte/Publications/fringe.pdf